TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

HCMUTE

BÁO CÁO ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

ĐỀ TÀI:

SMART LIBRARY - ỨNG DỤNG QUẢN LÝ THƯ VIỆN VÀ ĐẶT CHỖ NGỒI THÔNG MINH

Giảng viên hướng dẫn: Huỳnh Hoàng Hà

Nhóm thực hiện: Nhóm 8

Họ và tên:	MSSV:
Trần Chí Nguyên	22119179
Lê Xuân Trung	22119219
Đỗ Nguyễn Thảo Trang	23119214
Phan Hoàng Trí	23119218
Nguyễn Viết Nam	23119173
Trần Ngọc Minh	23119170

TP. Hồ Chí Minh, năm 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Th.S. Huỳnh Hoàng Hà. Trong suốt chặng đường nghiên cứu và thực hiện project môn học, thầy đã luôn dành thời gian quý báu để tận tâm định hướng, chỉ bảo và tạo mọi điều kiện tốt nhất giúp nhóm hoàn thành đề tài một cách trọn vẹn.

Bên cạnh đó, nhóm cũng xin gửi lời cảm ơn đến những người bạn đã nhiệt tình hỗ trợ, chia sẻ tài liệu và cùng nhau trao đổi kiến thức chuyên môn để nhóm có thể hoàn thành tốt đề tài của mình.

Mặc dù đã nỗ lực hết sức và dồn nhiều tâm huyết, song do những giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, đồ án chắc chắn vẫn khó tránh khỏi những sai sót nhất định. Vì vậy, nhóm kính mong nhận được những lời nhận xét, đánh giá và góp ý quý báu từ quý thầy/cô. Những ý kiến đóng góp này sẽ giúp các thành viên nhóm nhìn nhận lại vấn đề, trau dồi bản thân và áp dụng cho công việc thực tế trong tương lai.

Sau cùng, nhóm thực hiện đề tài xin kính chúc quý thầy/cô luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Xin chân thành cảm ơn!

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN 1
1.1 Giới thiệu dự án 1
1.2 Mục tiêu dự án 1
1.3 Phạm vi và đối tượng sử dụng 2
1.3.1 Phạm vi dự án 2
1.3.2 Đối tượng sử dụng 3
1.4 Chức năng chính của hệ thống 3
1.5 Kiến trúc tổng thể hệ thống 4
1.5.1 Tầng phần cứng – Library Hardware (ESP32) 5
1.5.2 Tầng đám mây – Firebase Cloud Backend 6
1.5.3 Tầng ứng dụng di động – Mobile Application 6
1.5.4 Module AI (Tùy chọn) 6
1.6 Công nghệ và công cụ sử dụng 6
1.7 Quy trình triển khai dự án 7
1.8 Cấu trúc báo cáo 9
1.9 Kết chương 10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 11
2.1 Nền tảng phát triển ứng dụng di động (React Native & TypeScript) 11
2.1.1 Tổng quan về công nghệ Cross-Platform và React Native 11
2.1.2 Ngôn ngữ TypeScript trong ứng dụng di động 12
2.1.3 Quản lý trạng thái ứng dụng với Redux Toolkit 13

2.1.4 Điều hướng và Quản lý màn hình với React Navigation 13
2.2 Nền tảng Điện toán Đám mây & Dịch vụ Backend (Google Firebase) 14
2.2.1 Firebase Authentication 14
2.2.2 Cơ sở dữ liệu Firebase Firestore NoSQL 14
2.2.3 Firebase Cloud Storage & Cloud Messaging (FCM) 15
2.2.4 Firebase Cloud Functions (Serverless Architecture) 15
2.3 Internet of Things (IoT) & Vi điều khiển ESP32 15
2.3.1 Tổng quan về vi điều khiển ESP32 15
2.3.2 Các cảm biến thu thập dữ liệu thực tế 16
2.3.3 Giao thức truyền thông IoT: MQTT vs HTTP/HTTPS 16
2.4 Trí tuệ Nhân tạo (AI) & Máy học trong Thư viện (Tùy chọn) 17
2.4.1 Hệ thống Gợi ý Sách thông minh (Recommendation System) 17
2.4.2 Mô hình Dự đoán Nhu cầu Sử dụng Chỗ ngồi 17
2.5 Bảng so sánh và đánh giá lựa chọn công nghệ 17
2.6 Kết chương 19
CHƯƠNG 3: PHÂN TÍCH KIẾN TRÚC VÀ CẤU TRÚC ỨNG DỤNG DI ĐỘNG SMARTLIBRARY 20
3.1 Tổng quan kiến trúc ứng dụng 20
3.2 Cấu trúc source code 21
3.2.1 Nhóm màn hình trong src/app 22
3.2.2 Nhóm context, config và types 23
3.2.3 Nhóm component quản trị 24
3.3 Phân tích các thành phần kiến trúc chính 24
3.3.1 Presentation Layer 24

3.3.2 State và Business Logic 25
3.3.3 Data Access Layer 25
3.4 Cơ chế điều hướng giữa các màn hình 26
3.5 Phân tích luồng xử lý các chức năng chính 27
3.5.1 Luồng đăng nhập và phân quyền 27
3.5.2 Luồng đăng ký tài khoản 28
3.5.3 Luồng tìm kiếm và xem chi tiết sách 28
3.5.4 Luồng mượn sách 28
3.5.5 Luồng quét mã sách 29
3.6 Giao tiếp giữa ứng dụng và Firebase 29
3.7 Đánh giá kiến trúc hiện tại 30
3.8 Kết luận chương 31
4.1 Mục tiêu và phạm vi đánh giá 32
4.1.1 Mục tiêu đánh giá 32
4.1.2 Cơ sở và phương pháp đánh giá 33
4.2 Kết quả triển khai các chức năng chính 33
4.2.1 Đăng nhập, đăng ký và phân quyền 34
4.2.2 Tìm kiếm sách và xem chi tiết 39
4.2.3 Mượn sách và cập nhật số lượng 41
4.2.4 Trả sách và lịch sử mượn 43
4.2.5 Quét barcode/QR bằng camera 45
4.2.6 Khu vực quản trị thư viện 47
4.3 Đánh giá dữ liệu và cơ chế đồng bộ 55
4.3.1 Vai trò của Firebase Authentication và Realtime Database 55

4.3.2 Tính nhất quán và an toàn dữ liệu 55
4.4 Đánh giá hiệu năng và trải nghiệm người dùng 56
4.4.1 Hiệu năng giao diện và tìm kiếm 56
4.4.2 Độ trễ đồng bộ dữ liệu 56
4.4.3 Độ ổn định và khả năng phục hồi 57
4.4.4 Bộ chỉ số cần đo trong thực nghiệm 57
4.5 Kế hoạch kiểm thử chức năng chi tiết 58
4.6 Đối chiếu với mục tiêu ban đầu 60
4.7 Hạn chế phát hiện và hướng cải thiện 62
4.7.1 Hạn chế về kiến trúc phần mềm 62
4.7.2 Hạn chế về độ tin cậy của giao dịch 62
4.7.3 Hạn chế về bảo mật 62
4.7.4 Hạn chế về bằng chứng kiểm thử và hiệu năng 63
4.7.5 Hạn chế về phạm vi IoT và AI 63
4.7.6 Lộ trình cải thiện theo mức ưu tiên 63
4.8 Kết luận chương 64
CHƯƠNG 5: KẾT LUẬN 65
5.1 Tổng kết những kết quả đạt được 65
5.2 Những hạn chế hiện tại 65
5.3 Hướng phát triển tương lai 66
LINK GITHUB VÀ VIDEO DEMO 68

DANH MỤC HÌNH ẢNH

Hình 1.1. Giao diện các chức năng chính của ứng dụng Smart Library (Mobile) 3
Hình 1.2. Kiến trúc tổng thể hệ thống Smart Library Management App 5
Hình 1.3. Quy trình triển khai dự án Smart Library (9 giai đoạn) 8
Hình 2.1. Kiến trúc luồng thực thi và Bridge/JSI trong React Native 12
Hình 3.1. Kiến trúc tổng thể ứng dụng SmartLibrary 26
Hình 3.2. Sơ đồ điều hướng chính của ứng dụng 27
Hình 3.3. Sequence Diagram chức năng mượn sách 29
Hình 4.1. Giao diện đăng nhập của độc giả và quản thư 35
Hình 4.2. Giao diện đăng ký của ứng dụng *36*
Hình 4.3. Thông tin tài khoản email của ứng dụng *37*
Hình 4.4. Giao diện chính của ứng dụng độc giả *39*
Hình 4.5. Giao diện chính ứng dụng của bên quản thư *39*
Hình 4.6. Danh sách sách của thư viện *40*
Hình 4.7. Yêu cầu mượn sách được thực hiện *42*
Hình 4.8. Lịch sử mượn sách *44*
Hình 4.9. Giao diện quét mã sách bằng camera *46*
Hình 4.10. Giao diện quản lý danh mục sách và tài khoản người dùng *48*
Hình 4.11. Giao diện quản lý thẻ RFID, mượn sách, trả sách và sách quá hạn *50*
Hình 4.12. Giao diện gửi thông báo và thống kê tình trạng thư viện *52*

DANH MỤC BẢNG

Bảng 1.1. Công nghệ và công cụ sử dụng trong dự án Smart Library 7

Bảng 2.1. So sánh các giải pháp phát triển ứng dụng di động 18

Bảng 2.2. So sánh giải pháp cơ sở dữ liệu Backend 19

Bảng 2.3. So sánh các giao thức truyền dữ liệu IoT 19

CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN

1.1 Giới thiệu dự án

Smart Library là một hệ thống thư viện thông minh tích hợp công nghệ hiện đại, được phát triển nhằm nâng cao trải nghiệm học tập và tra cứu tài liệu cho sinh viên, giảng viên và cán bộ trong môi trường đại học. Dự án ra đời xuất phát từ thực tế rằng các thư viện truyền thống thường gặp khó khăn trong việc quản lý tài nguyên, theo dõi tình trạng chỗ ngồi, và cung cấp thông tin kịp thời đến người dùng.

Hệ thống Smart Library kết hợp ba thành phần chính: ứng dụng di động (Mobile App) phát triển bằng React Native và TypeScript, phần cứng IoT (ESP32) để thu thập dữ liệu thực tế trong thư viện, và nền tảng đám mây Firebase để đồng bộ và lưu trữ dữ liệu theo thời gian thực. Ngoài ra, hệ thống còn tích hợp module AI (tùy chọn) để gợi ý sách và dự đoán nhu cầu chỗ ngồi.

Với khẩu hiệu "Đọc sách mọi lúc, mọi nơi – Kết nối tri thức – Kiến tạo tương lai", Smart Library hướng tới một môi trường học tập hiện đại, thông minh và tiện lợi, nơi mà người dùng có thể dễ dàng tìm kiếm sách, đặt chỗ ngồi, mượn/trả tài liệu và theo dõi lịch sử sử dụng chỉ thông qua chiếc điện thoại thông minh.

1.2 Mục tiêu dự án

Dự án Smart Library được xây dựng với các mục tiêu cụ thể như sau:

- **Mục tiêu kỹ thuật:** Phát triển ứng dụng di động đa nền tảng (Android/iOS) sử dụng React Native và TypeScript, tích hợp với phần cứng ESP32 qua giao thức MQTT/HTTP và nền tảng Firebase Cloud.

- **Mục tiêu chức năng:** Cung cấp đầy đủ các chức năng quản lý thư viện bao gồm đăng nhập, tìm kiếm sách, xem chi tiết sách, đặt chỗ ngồi theo thời gian thực, mượn/trả sách, xem lịch sử và thống kê cá nhân.
- **Mục tiêu trải nghiệm người dùng:** Thiết kế giao diện thân thiện, trực quan và dễ sử dụng; gửi thông báo kịp thời về tình trạng sách và chỗ ngồi; cá nhân hóa trải nghiệm theo sở thích đọc sách.
- **Mục tiêu nghiên cứu:** Nghiên cứu và áp dụng mô hình AI vào việc gợi ý sách phù hợp và dự đoán nhu cầu sử dụng phòng đọc, từ đó nâng cao hiệu quả quản lý tài nguyên thư viện.
- **Mục tiêu học thuật:** Áp dụng kiến thức về lập trình di động, IoT, điện toán đám mây và trí tuệ nhân tạo vào một sản phẩm thực tế có giá trị ứng dụng cao trong môi trường đại học.

1.3 Phạm vi và đối tượng sử dụng

1.3.1 Phạm vi dự án

Dự án tập trung phát triển hệ thống thư viện thông minh áp dụng trong phạm vi thư viện của một trường đại học, với các phạm vi cụ thể:

- Phạm vi phần mềm: Ứng dụng di động (React Native + TypeScript) chạy trên Android và iOS; backend sử dụng hoàn toàn Firebase (Authentication, Firestore, Storage, Cloud Functions, Cloud Messaging).
- Phạm vi phần cứng: Bo mạch ESP32 kết nối WiFi, cảm biến ghế (PIR/Weight), cảm biến nhiệt độ - độ ẩm (DHT22), cảm biến ánh sáng (LDR), đầu đọc RFID tùy chọn (MFRC522).
- Phạm vi AI: Module AI tùy chọn, phân tích dữ liệu lịch sử mượn sách và thói quen người dùng để gợi ý sách và dự đoán nhu cầu chỗ ngồi.
- Phạm vi triển khai: Hệ thống được thiết kế cho quy mô thư viện vừa và nhỏ, có thể mở rộng (scale) khi cần thiết.

1.3.2 Đối tượng sử dụng

- **Sinh viên:** Đối tượng chính của hệ thống. Sử dụng ứng dụng để tìm kiếm sách, đặt chỗ ngồi, mượn/trả tài liệu và nhận thông báo từ thư viện.
- **Giảng viên và cán bộ:** Sử dụng hệ thống để tra cứu tài liệu, đặt phòng học/khu vực đọc sách và theo dõi tình trạng tài nguyên.
- **Quản trị viên thư viện:** Quản lý danh mục sách, tài khoản người dùng, giám sát trạng thái phần cứng và xem báo cáo thống kê tổng hợp.

1.4 Chức năng chính của hệ thống

Hệ thống Smart Library được thiết kế với đầy đủ các chức năng cần thiết cho một thư viện thông minh hiện đại. Dưới đây là mô tả chi tiết các chức năng chính của ứng dụng di động:



Hình 1.1. Giao diện các chức năng chính của ứng dụng Smart Library (Mobile)

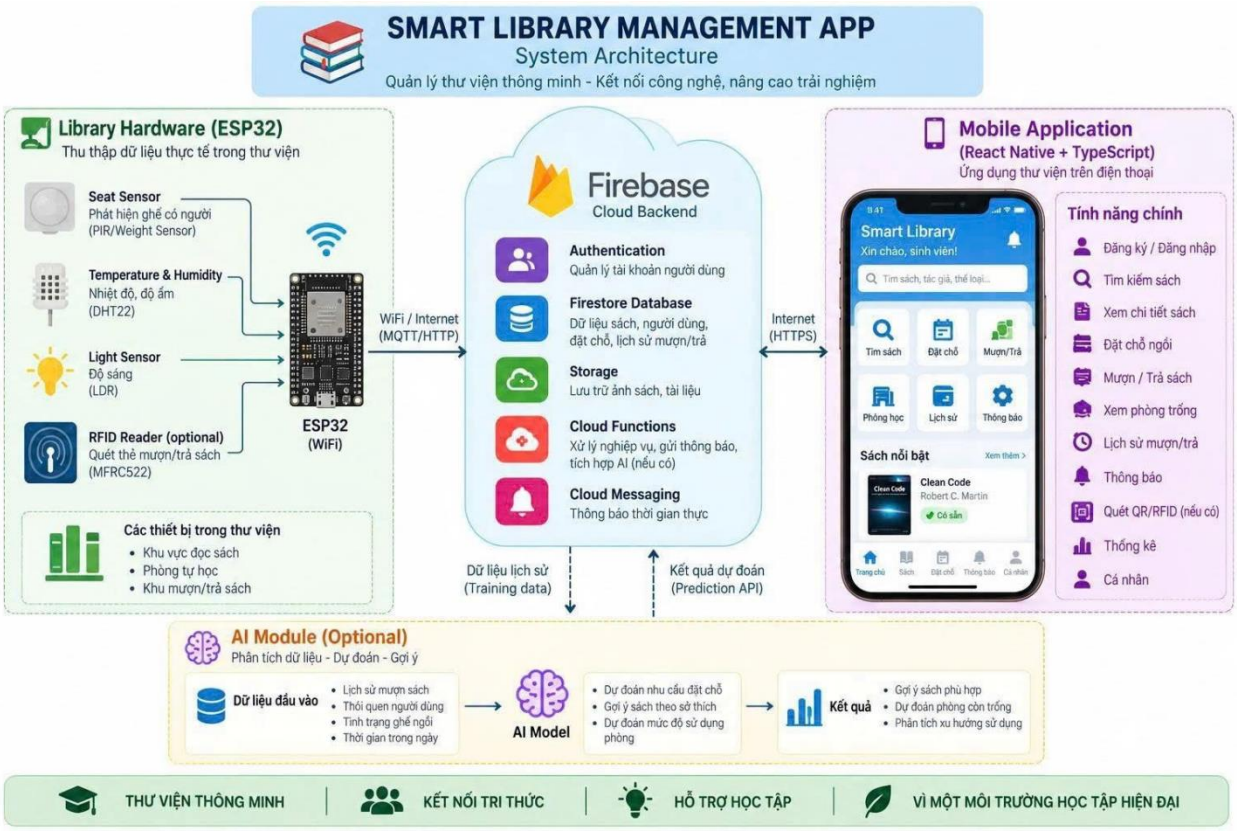
Ứng dụng Smart Library trên di động bao gồm 7 màn hình chức năng chính:

- **Đăng nhập:** Hỗ trợ đăng nhập bằng email/mật khẩu hoặc tài khoản Google/Apple. Cho phép người dùng tạo tài khoản mới nếu chưa có.
- **Trang chủ:** Hiển thị lời chào cá nhân hóa, các tính năng chính (Tìm sách, Đặt sách, Đặt chỗ ngồi, Lịch sử, Thông báo, Tin tức), sách nổi bật và gợi ý sách.
- **Tìm kiếm sách:** Hỗ trợ tìm kiếm theo tên sách, tác giả, thể loại với bộ lọc thông minh. Hiển thị kết quả với thông tin vị trí (kệ, hàng) trong thư viện.
- **Chi tiết sách:** Cung cấp thông tin đầy đủ về sách bao gồm ảnh bìa, mô tả, số lượng còn lại, vị trí lưu trữ và chức năng đặt mượn hoặc đọc thử.
- **Đặt chỗ ngồi:** Hiển thị sơ đồ tầng theo thời gian thực với trạng thái từng ghế (trống/đã đặt/đang sử dụng), cho phép người dùng chọn và đặt chỗ.
- **Lịch sử:** Lưu trữ và hiển thị lịch sử sách đã mượn cùng hạn trả, lịch sử đặt chỗ ngồi, giúp người dùng theo dõi hoạt động cá nhân.
- **Thống kê:** Cung cấp biểu đồ trực quan về số sách đã mượn theo thời gian, thể loại sách yêu thích, thời gian đọc trung bình và tổng số sách đã đọc.

Ngoài ra, hệ thống còn hỗ trợ các tính năng bổ sung như: thông báo đẩy (sách đến hạn, xác nhận đặt chỗ), thống kê sử dụng, và cài đặt cá nhân hóa giao diện.

1.5 Kiến trúc tổng thể hệ thống

Kiến trúc hệ thống Smart Library được thiết kế theo mô hình ba tầng (three-tier), bao gồm tầng phần cứng (Hardware Layer), tầng dịch vụ đám mây (Cloud Layer) và tầng ứng dụng di động (Application Layer), được bổ sung thêm module AI tùy chọn.



Hình 1.2. Kiến trúc tổng thể hệ thống Smart Library Management App

1.5.1 Tầng phần cứng – Library Hardware (ESP32)

ESP32 đóng vai trò là bộ xử lý trung tâm của tầng phần cứng, thu thập dữ liệu thực tế trong thư viện từ các cảm biến và truyền lên Firebase thông qua giao thức WiFi/Internet (MQTT hoặc HTTP). Các thiết bị phần cứng bao gồm:

- Seat Sensor (PIR/Weight Sensor): Phát hiện ghế có người ngồi hay không.
- Temperature & Humidity (DHT22): Theo dõi nhiệt độ và độ ẩm trong phòng đọc sách.
- Light Sensor (LDR): Đo cường độ ánh sáng để tự động điều chỉnh thiết bị chiếu sáng.
- RFID Reader – MFRC522 (tùy chọn): Quét mã RFID để xác thực mượn/trả sách tự động.

1.5.2 Tầng đám mây – Firebase Cloud Backend

Firebase được chọn làm nền tảng backend chính nhờ khả năng cung cấp dữ liệu theo thời gian thực, bảo mật cao và dễ tích hợp. Các dịch vụ Firebase được sử dụng bao gồm:

- Authentication: Xác thực và quản lý tài khoản người dùng.
- Firestore Database: Lưu trữ dữ liệu sách, người dùng, lịch đặt chỗ, lịch sử mượn/trả.
- Storage: Lưu trữ ảnh bìa sách và các tài liệu đính kèm.
- Cloud Functions: Xử lý nghiệp vụ phức tạp, gửi thông báo tự động, tích hợp API AI.
- Cloud Messaging (FCM): Gửi thông báo đẩy theo thời gian thực đến người dùng.

1.5.3 Tầng ứng dụng di động – Mobile Application

Ứng dụng di động được phát triển bằng React Native kết hợp TypeScript, cho phép triển khai đồng thời trên cả hai nền tảng Android và iOS từ một codebase duy nhất. Giao tiếp với Firebase thông qua giao thức HTTPS bảo mật.

1.5.4 Module AI (Tùy chọn)

Module AI phân tích dữ liệu lịch sử (lịch sử mượn sách, thói quen người dùng, tình trạng ghế, thời gian trong ngày) để đưa ra các dự đoán và gợi ý thông minh:

- Gợi ý sách phù hợp với sở thích và lịch sử đọc của từng người dùng.
- Dự đoán nhu cầu đặt chỗ ngồi theo khung giờ và khu vực trong thư viện.
- Phân tích xu hướng sử dụng thư viện để hỗ trợ ban quản lý ra quyết định.

1.6 Công nghệ và công cụ sử dụng

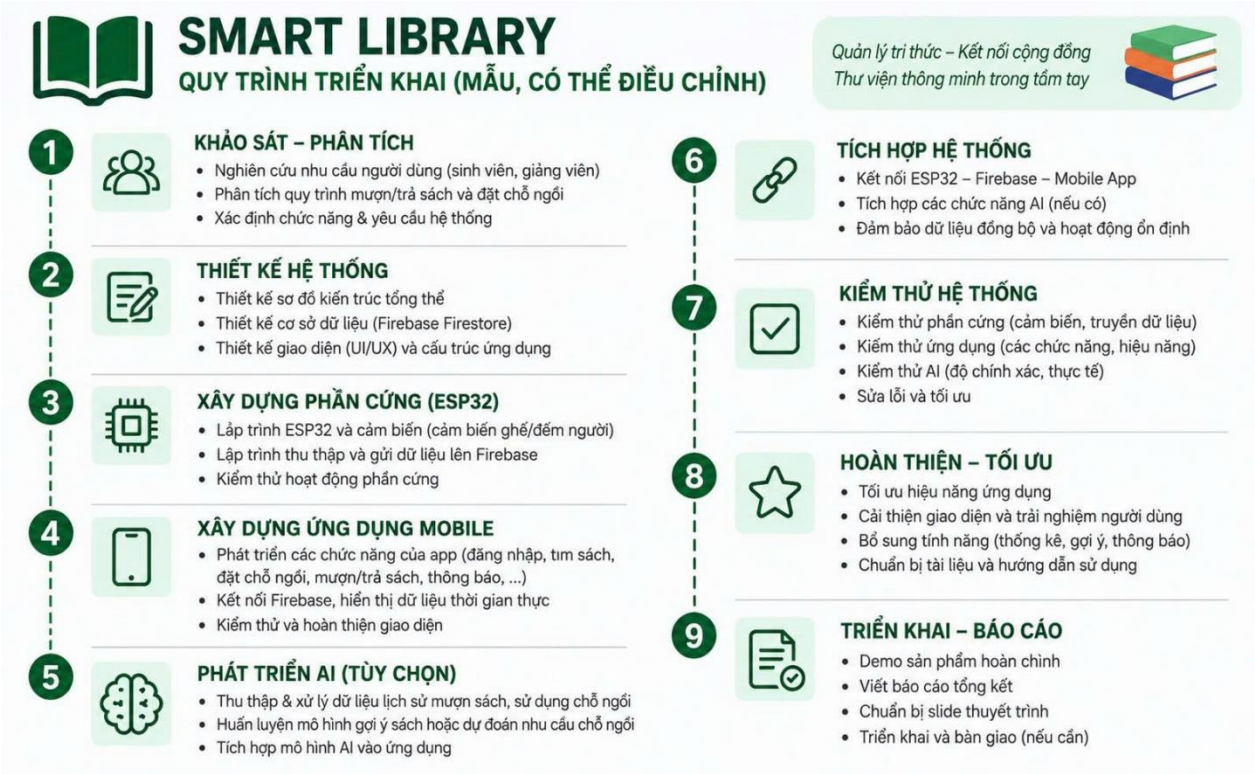
Hệ thống Smart Library được xây dựng trên nền tảng các công nghệ hiện đại, phù hợp với xu hướng phát triển phần mềm trong thời đại số. Bảng 1.1 tóm tắt các công nghệ và công cụ chính được sử dụng trong dự án:

Thành phần	Công nghệ / Công cụ	Mô tả
Ứng dụng di động	React Native + TypeScript	Framework đa nền tảng (Android & iOS)
Backend / Cloud	Firebase (Google)	Authentication, Firestore, Storage, FCM, Functions
Phần cứng IoT	ESP32 (WiFi)	Vi điều khiển thu thập và gửi dữ liệu cảm biến
Cảm biến	PIR, DHT22, LDR, MFRC522	Phát hiện người, đo môi trường, quét RFID
Truyền thông	MQTT / HTTP / HTTPS	Giao thức kết nối ESP32 – Firebase – App
AI (tùy chọn)	Python ML Models + Prediction API	Gợi ý sách, dự đoán nhu cầu chỗ ngồi
Quản lý phiên bản	Git / GitHub	Kiểm soát mã nguồn và cộng tác nhóm
Thiết kế UI/UX	Figma	Thiết kế wireframe và prototype giao diện

Bảng 1.1. Công nghệ và công cụ sử dụng trong dự án Smart Library

1.7 Quy trình triển khai dự án

Dự án Smart Library được triển khai theo một quy trình có cấu trúc rõ ràng, bao gồm 9 giai đoạn chính từ khảo sát ban đầu đến triển khai và báo cáo kết quả. Quy trình này đảm bảo tính hệ thống và chất lượng của sản phẩm cuối cùng.



Hình 1.3. Quy trình triển khai dự án Smart Library (9 giai đoạn)

Chi tiết 9 giai đoạn trong quy trình triển khai dự án:

- **Giai đoạn 1 – Khảo sát & Phân tích:** Nghiên cứu nhu cầu người dùng (sinh viên, giảng viên); phân tích quy trình mượn/trả sách và đặt chỗ ngồi hiện tại; xác định chức năng và yêu cầu hệ thống.
- **Giai đoạn 2 – Thiết kế hệ thống:** Thiết kế sơ đồ kiến trúc tổng thể; thiết kế cơ sở dữ liệu Firebase Firestore; thiết kế giao diện UI/UX và cấu trúc ứng dụng bằng Figma.
- **Giai đoạn 3 – Xây dựng phần cứng (ESP32):** Lập trình ESP32 và kết nối cảm biến ghế, cảm biến môi trường; lập trình thu thập và gửi dữ liệu lên Firebase; kiểm thử hoạt động phần cứng.
- **Giai đoạn 4 – Xây dựng ứng dụng Mobile:** Phát triển các màn hình chức năng (đăng nhập, tìm sách, đặt chỗ, mượn/trả, thông báo); kết nối Firebase và hiển thị dữ liệu thời gian thực; kiểm thử và hoàn thiện giao diện.

- Giai đoạn 5 – Phát triển AI (Tùy chọn): Thu thập và xử lý dữ liệu lịch sử mượn sách, sử dụng chỗ ngồi; huấn luyện mô hình gợi ý sách và dự đoán nhu cầu; tích hợp module AI vào ứng dụng.

- Giai đoạn 6 – Tích hợp hệ thống: Kết nối ESP32 – Firebase – Mobile App thành một hệ thống hoàn chỉnh; tích hợp các chức năng AI; đảm bảo dữ liệu đồng bộ và hoạt động ổn định.

- Giai đoạn 7 – Kiểm thử hệ thống: Kiểm thử phần cứng (cảm biến, truyền dữ liệu); kiểm thử ứng dụng (các chức năng, hiệu năng); kiểm thử AI (độ chính xác, thực tế); sửa lỗi và tối ưu hóa.

- Giai đoạn 8 – Hoàn thiện & Tối ưu: Tối ưu hiệu năng ứng dụng; cải thiện giao diện và trải nghiệm người dùng; bổ sung tính năng thống kê, gợi ý, thông báo; chuẩn bị tài liệu hướng dẫn.

- Giai đoạn 9 – Triển khai & Báo cáo: Demo sản phẩm hoàn chỉnh; viết báo cáo tổng kết theo đúng chuẩn yêu cầu; chuẩn bị slide thuyết trình; triển khai và bàn giao sản phẩm (nếu cần).

1.8 Cấu trúc báo cáo

Báo cáo đồ án môn học Phát triển Ứng dụng Di động – Smart Library được tổ chức thành 5 chương như sau:

Chương 1 – Tổng quan về dự án: Giới thiệu tổng quan về Smart Library, mục tiêu, phạm vi, đối tượng sử dụng, kiến trúc hệ thống, công nghệ sử dụng và quy trình triển khai.

Chương 2 – Cơ sở lý thuyết: Trình bày các kiến thức nền tảng về React Native, Firebase, IoT với ESP32, giao thức MQTT/HTTP, và các thuật toán AI được áp dụng trong dự án.

Chương 3 – Phân tích và thiết kế hệ thống: Phân tích yêu cầu chức năng và phi chức năng; thiết kế cơ sở dữ liệu, kiến trúc chi tiết, sơ đồ use case, sequence diagram và giao diện người dùng.

Chương 4 – Phân tích kết quả: Trình bày kết quả thực nghiệm, đánh giá hiệu năng hệ thống, kết quả kiểm thử và so sánh với yêu cầu ban đầu.

Chương 5 – Kết luận: Tổng kết những gì đã đạt được, hạn chế hiện tại và hướng phát triển tương lai của dự án Smart Library.

1.9 Kết chương

Chương 1 đã trình bày tổng quan về dự án Smart Library – hệ thống thư viện thông minh tích hợp ứng dụng di động, phần cứng IoT và nền tảng đám mây. Qua chương này, người đọc có thể nắm được bức tranh tổng thể về dự án: từ mục tiêu, phạm vi, đối tượng sử dụng đến kiến trúc kỹ thuật và lộ trình triển khai.

Với sự kết hợp giữa React Native, Firebase và ESP32, Smart Library hứa hẹn mang lại một giải pháp quản lý thư viện toàn diện, hiện đại và tiện lợi. Các chương tiếp theo sẽ đi sâu vào cơ sở lý thuyết, thiết kế chi tiết, kết quả thực nghiệm và kết luận của dự án.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

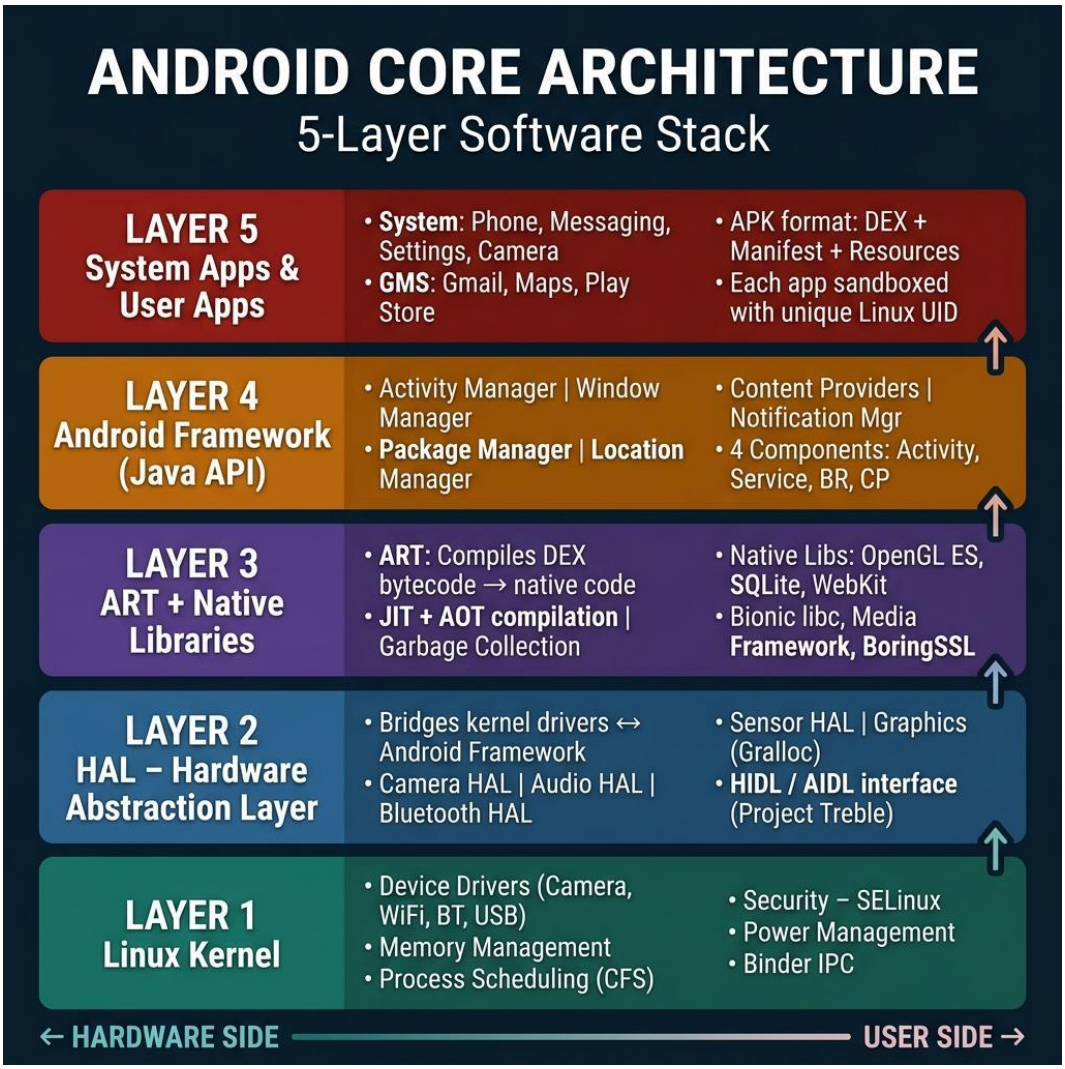
Chương này trình bày toàn bộ cơ sở lý thuyết, các nền tảng công nghệ, giao thức truyền thông và mô hình kiến trúc cốt lõi được ứng dụng trong hệ thống Smart Library. Nắm vững các kiến thức này là tiền đề quan trọng để phân tích, thiết kế và triển khai ứng dụng di động cũng như các thành phần IoT và Backend đám mây.

2.1 Nền tảng phát triển ứng dụng di động (React Native & TypeScript)

2.1.1 Tổng quan về công nghệ Cross-Platform và React Native

Trong phát triển ứng dụng di động hiện đại, hai tiếp cận phổ biến nhất là Native Development (viết mã nguồn riêng biệt cho Android bằng Java/Kotlin và iOS bằng Swift/Objective-C) và Cross-Platform Development (sử dụng một codebase duy nhất để biên dịch hoặc chạy trên nhiều nền tảng). React Native là một framework mã nguồn mở do Meta (Facebook) phát triển, cho phép xây dựng ứng dụng di động thật sự (native mobile apps) sử dụng JavaScript và React.

Điểm khác biệt cốt lõi của React Native so với các giải pháp Web-view (như PhoneGap/Ionic) là React Native không renders HTML trong WebView, mà render trực tiếp ra các thành phần giao diện gốc (Native UI Components) của HĐH Android và iOS. Nhờ đó, ứng dụng đạt được hiệu năng mượt mà, phản hồi giao diện tức thì và giữ được trải nghiệm người dùng (look and feel) tự nhiên của từng hệ điều hành.



Hình 2.1. Kiến trúc luồng thực thi và Bridge/JSI trong React Native

2.1.2 Ngôn ngữ TypeScript trong ứng dụng di động

TypeScript là một siêu ngôn ngữ (superset) của JavaScript do Microsoft phát triển, bổ sung hệ thống kiểu tĩnh (static typing) và các tính năng hướng đối tượng nâng cao. Việc kết hợp TypeScript với React Native mang lại những lợi ích vượt trội cho dự án Smart Library:

- **Kiểm soát kiểu chặt chẽ:** Phát hiện lỗi ngay tại thời điểm biên dịch (Compile-time type checking), tránh các lỗi nguy hiểm phổ biến trong JavaScript như `TypeError: Cannot read property of undefined`.

- **Mô hình hóa dữ liệu chính xác:** Định nghĩa rõ ràng cấu trúc dữ liệu cho Sách (Book), Chỗ ngồi (Seat), Người dùng (User), Giao dịch mượn (BorrowingTransaction) thông qua Interface và Type Aliases.
- **Hỗ trợ công cụ lập trình:** IntelliSense và Auto-complete mạnh mẽ trên các IDE như VS Code giúp tăng tốc độ lập trình và giảm sai sót tên thuộc tính.
- **Tái cấu trúc mã nguồn an toàn:** Dễ dàng bảo trì và mở rộng hệ thống khi dự án phát triển lớn hơn hoặc khi làm việc theo nhóm.

2.1.3 Quản lý trạng thái ứng dụng với Redux Toolkit

Đối với các ứng dụng có luồng dữ liệu phức tạp như Smart Library (quản lý trạng thái đăng nhập, danh mục sách, sơ đồ chỗ ngồi thời gian thực, giỏ mượn sách, cài đặt người dùng), việc quản lý state cục bộ tại từng Component sẽ dẫn đến tình trạng "prop drilling" và khó đồng bộ dữ liệu. Dự án sử dụng Redux Toolkit – thư viện chuẩn chính thức để quản lý state tập trung.

Kiến trúc Redux trong ứng dụng bao gồm 4 thành phần chính:

- **Single Store:** Nơi chứa toàn bộ cây trạng thái (State Tree) của ứng dụng.
- **Slices:** Chia nhỏ State và Reducer theo từng module chức năng (ví dụ: `authSlice`, `bookSlice`, `seatSlice`).
- **Reducers:** Hàm nguyên tố (Pure Function) chịu trách nhiệm tính toán State mới dựa trên Action nhận được.
- **createAsyncThunk:** Xử lý các tác vụ bất đồng bộ như gọi API Firebase (Fetch danh sách sách, gửi đăng ký chỗ ngồi) trước khi cập nhật dữ liệu vào Store.

2.1.4 Điều hướng và Quản lý màn hình với React Navigation

React Navigation là thư viện tiêu chuẩn để quản lý luồng chuyển cảnh và điều hướng trong React Native. Hệ thống Smart Library kết hợp hai bộ điều hướng chính:

Native Stack Navigator: Quản lý chuyển cảnh phân cấp dạng chồng xếp, sử dụng cho luồng Đăng nhập/Đăng ký (`AuthStack`) và các màn hình chi tiết như Chi tiết sách (`DetailsScreen`), Quét mã (`CameraScreen`).

Bottom Tabs Navigator: Hiển thị thanh menu chính phía dưới màn hình (`MainTab`), cho phép người dùng chuyển đổi nhanh giữa Trang chủ (`HomeScreen`), Đặt chỗ (`SeatBookingScreen`), Sách của tôi (`MyBooksScreen`) và Cài đặt (`SettingsScreen`).

2.2 Nền tảng Điện toán Đám mây & Dịch vụ Backend (Google Firebase)

Google Firebase là nền tảng điện toán đám mây theo mô hình Backend-as-a-Service (BaaS), cung cấp cơ sở hạ tầng và dịch vụ backend sẵn có giúp các nhà phát triển tập trung tối đa vào trải nghiệm người dùng trên thiết bị di động. Các dịch vụ Firebase cốt lõi được tích hợp trong hệ thống Smart Library bao gồm:

2.2.1 Firebase Authentication

Cung cấp dịch vụ quản lý tài khoản và xác thực người dùng an toàn với tiêu chuẩn công nghiệp. Hỗ trợ đăng nhập đa dạng: Email/Mật khẩu mã hóa, Đăng nhập một chạm (Single Sign-On) qua tài khoản Google hoặc Apple. Firebase Auth tự động quản lý phiên làm việc (Session Management) và cấp phát mã thông báo JWT (JSON Web Token) để bảo mật các yêu cầu truy vấn dữ liệu.

2.2.2 Cơ sở dữ liệu Firebase Firestore NoSQL

Cloud Firestore là cơ sở dữ liệu NoSQL linh hoạt, mở rộng cao lưu trữ dữ liệu dưới dạng các Document (Tài liệu) nằm trong các Collection (Tập hợp). Đặc tính quan trọng nhất của Firestore đối với dự án là tính năng Real-time Data Synchronization (Đồng bộ dữ liệu thời gian thực).

Thông qua cơ chế WebSocket và lắng nghe sự kiện `onSnapshot`, khi trạng thái một chiếc ghế trong thư viện thay đổi (từ trống sang có người ngồi do cảm biến ESP32 gửi lên), Firestore sẽ tự động đẩy (push) cập nhật mới nhất đến tất cả các ứng dụng di động đang mở màn hình sơ đồ chỗ ngồi mà không cần người dùng phải làm mới (refresh) trang.

2.2.3 Firebase Cloud Storage & Cloud Messaging (FCM)

Cloud Storage: Lưu trữ các tệp tin có dung lượng lớn như ảnh bìa sách chất lượng cao, tệp PDF xem trước tài liệu và ảnh đại diện người dùng với tốc độ tải nhanh nhờ hệ thống CDN của Google.

Firebase Cloud Messaging (FCM): Dịch vụ gửi thông báo đẩy (Push Notification) cross-platform. FCM cho phép ứng dụng gửi thông báo nhắc nhở sắp đến hạn trả sách, thông báo xác nhận đặt chỗ thành công ngay cả khi ứng dụng đang chạy ngầm hoặc đã đóng.

2.2.4 Firebase Cloud Functions (Serverless Architecture)

Cloud Functions cho phép chạy mã backend Node.js tự động trên hạ tầng serverless của Google để phản hồi các sự kiện từ Firestore hoặc HTTPS requests. Các tác vụ serverless bao gồm: tự động quét và kiểm tra sách quá hạn lúc 00:00 hàng ngày, tự động hủy giữ chỗ ngồi nếu người dùng không đến xác nhận trong vòng 15 phút, và tính toán điểm uy tín mượn sách.

2.3 Internet of Things (IoT) & Vi điều khiển ESP32

2.3.1 Tổng quan về vi điều khiển ESP32

ESP32 là dòng vi điều khiển SoC (System on a Chip) giá thành thấp, năng lượng thấp do Espressif Systems sản xuất. ESP32 sở hữu vi xử lý lõi đôi Tensilica Xtensa 32-bit LX6 với xung nhịp lên tới 240 MHz, tích hợp sẵn kết nối Wi-Fi 802.11 b/g/n và Bluetooth

4.2 / BLE. Đây là bộ não trung tâm điều khiển toàn bộ thiết bị IoT trong phòng đọc thư viện.

2.3.2 Các cảm biến thu thập dữ liệu thực tế

Để hiện thực hóa tính năng "Thư viện Thông minh", tầng phần cứng tích hợp nhiều loại cảm biến chuyên dụng:

- **Cảm biến phát hiện có người (PIR Sensor):** Sử dụng tia hồng ngoại thụ động để phát hiện chuyển động/sự có mặt của sinh viên tại từng vị trí ghế ngồi.
- **Cảm biến trọng lượng (Loadcell + HX711):** Kết hợp với mạch khuếch đại HX711 đặt dưới chân ghế để đo chính xác trọng lượng, xác nhận ghế đang thực sự được sử dụng hay chỉ đặt đồ đạc.
- **Cảm biến môi trường DHT22:** Đo nhiệt độ (độ chính xác ±0.5°C) và độ ẩm phòng đọc để ban quản lý điều chỉnh hệ thống điều hòa không khí hợp lý.
- **Cảm biến ánh sáng LDR:** Đo cường độ ánh sáng tại các khu vực bàn học để tự động điều chỉnh đèn đọc sách đạt tiêu chuẩn bảo vệ thị lực.
- **Đầu đọc thẻ RFID MFRC522:** Đầu đọc thẻ RFID tần số 13.56 MHz hỗ trợ sinh viên quét thẻ sinh viên/thẻ thư viện để tự động mở tủ mượn sách hoặc quẹt thẻ nhận chỗ ngồi.

2.3.3 Giao thức truyền thông IoT: MQTT vs HTTP/HTTPS

Truyền dữ liệu giữa vi điều khiển ESP32 và hệ thống Firebase Đám mây đóng vai trò quyết định độ trễ của hệ thống. Hai giao thức truyền thông phổ biến được xem xét:

- **Giao thức HTTP/HTTPS:** Giao thức truyền nhận theo mô hình Request/Response truyền thống trên nền TCP/IP. Mỗi lần gửi dữ liệu cảm biến, ESP32 phải thiết lập lại kết nối TLS/HTTPS handshake, tạo overhead lớn về bộ nhớ và tiêu tốn năng lượng.

- **Giao thức MQTT (Message Queuing Telemetry Transport):** Giao thức truyền tin siêu nhẹ theo mô hình Publish/Subscribe (Phát hành / Đăng ký) chạy trên nền TCP. MQTT được thiết kế tối ưu cho các thiết bị IoT băng thông thấp và tài nguyên hạn chế. ESP32 đóng vai trò Publisher gửi dữ liệu cảm biến lên MQTT Broker, và Cloud Service sẽ Subscribe để cập nhật vào Firestore.

2.4 Trí tuệ Nhân tạo (AI) & Máy học trong Thư viện (Tùy chọn)

2.4.1 Hệ thống Gợi ý Sách thông minh (Recommendation System)

Nhằm nâng cao trải nghiệm đọc sách và giúp sinh viên dễ dàng tiếp cận các tài liệu học tập phù hợp, hệ thống tích hợp hai kỹ thuật máy học cơ bản:

- **Content-Based Filtering (Lọc dựa trên nội dung):** Phân tích các đặc trưng của sách (Thể loại, Tác giả, Từ khóa nội dung) và so sánh với lịch sử mượn sách trước đây của sinh viên để đưa ra danh sách đề xuất tương đồng.
- **Collaborative Filtering (Lọc cộng tác):** Tìm kiếm các độc giả có thói quen đọc sách tương tự (User-based) hoặc các cuốn sách thường được mượn cùng nhau (Item-based) để gợi ý những cuốn sách sinh viên có thể yêu thích.

2.4.2 Mô hình Dự đoán Nhu cầu Sử dụng Chỗ ngồi

Áp dụng mô hình Hồi quy (Regression Model / Time Series Analysis) dựa trên các chuỗi dữ liệu lịch sử (khung giờ trong ngày, ngày trong tuần, thời điểm trước kỳ thi, điều kiện thời tiết) để dự đoán tỷ lệ lấp đầy chỗ ngồi. Tính năng này giúp sinh viên chủ động đặt chỗ trước và giúp ban quản lý tối ưu hóa điều hòa và chiếu sáng.

2.5 Bảng so sánh và đánh giá lựa chọn công nghệ

Dưới đây là các bảng tổng hợp và so sánh phân tích ưu/nhược điểm làm căn cứ khoa học cho việc lựa chọn công nghệ trong dự án Smart Library:

Tiêu chí	React Native	Flutter	Native Android (Kotlin)
Ngôn ngữ	TypeScript / JavaScript	Dart	Kotlin / Java
Hiệu năng UI	Rất cao (Native Components)	Rất cao (Skia Engine)	Tuyệt đối (Native)
Hệ sinh thái	Khổng lồ (npm, React Community)	Phát triển nhanh	Chính thức từ Google
Code Reuse	85-95% chung Android & iOS	90-95% chung Android & iOS	0% (Chỉ chạy Android)
Độ phức tạp	Vừa phải, tiếp cận nhanh	Học ngôn ngữ Dart mới	Tốn nhiều thời gian

Bảng 2.1. So sánh các giải pháp phát triển ứng dụng di động

Đặc tính	Firebase Firestore	Realtime Database	Traditional SQL (MySQL)
Mô hình dữ liệu	Document - Collection (NoSQL)	JSON Tree lớn (NoSQL)	Bảng quan hệ (Relational)
Đồng bộ thời gian thực	Rất mạnh (onSnapshot, Offline support)	Mạnh (Event listeners)	Cần Cấu hình WebSocket thủ công
Khả năng truy vấn	Mạnh (Filter, Compound queries)	Hạn chế (Chỉ lọc đơn)	Rất mạnh (SQL JOIN, Complex)
Mở rộng (Scalability)	Tự động mở rộng (Auto-scale)	Giới hạn theo connection	Cần quản lý Server / Cluster

Bảng 2.2. So sánh giải pháp cơ sở dữ liệu Backend

Tiêu chí	MQTT (Được chọn cho IoT)	HTTP / REST API	WebSocket
Mô hình truyền tin	Publish / Subscribe	Request / Response	Bidirectional Full-Duplex
Overhead gói tin	Rất nhỏ (Header 2 Bytes)	Lớn (HTTP Headers ~1KB)	Vừa phải
Tiêu thụ năng lượng	Rất thấp (Tối ưu cho Pin/ESP32)	Cao (Tạo kết nối lại)	Trung bình
Băng thông yêu cầu	Thấp (Thích hợp mạng yếu)	Cao	Vừa phải

Bảng 2.3. So sánh các giao thức truyền dữ liệu IoT

2.6 Kết chương

Chương 2 đã hệ thống hóa đầy đủ và toàn diện các cơ sở lý thuyết cốt lõi phục vụ cho việc xây dựng hệ thống Smart Library. Từ việc lựa chọn công nghệ React Native kết hợp TypeScript để xây dựng ứng dụng di động đa nền tảng mượt mà, sử dụng dịch vụ điện toán đám mây Google Firebase và Firestore để đồng bộ dữ liệu thời gian thực, đến việc làm chủ vi điều khiển ESP32 và các cảm biến IoT cùng giao thức MQTT siêu nhẹ. Bên cạnh đó, mô hình gợi ý AI và dự đoán chỗ ngồi cũng tạo ra điểm nhấn thông minh cho sản phẩm.

CHƯƠNG 3: PHÂN TÍCH KIẾN TRÚC VÀ CẤU TRÚC ỨNG DỤNG DI ĐỘNG SMARTLIBRARY

Chương này trình bày kiến trúc phần mềm, cấu trúc source code, cơ chế điều hướng, luồng dữ liệu và phương thức giao tiếp với Firebase của ứng dụng di động SmartLibrary. Nội dung được phân tích dựa trên source code hiện có của dự án, tập trung vào các thành phần thực tế được triển khai thay vì gán một mô hình kiến trúc không tồn tại trong code.

3.1 Tổng quan kiến trúc ứng dụng

SmartLibrary được phát triển bằng React Native kết hợp Expo và TypeScript. Đây là mô hình phát triển ứng dụng đa nền tảng, cho phép sử dụng chung phần lớn source code cho Android, iOS và Web. Ứng dụng sử dụng Expo Router làm cơ chế điều hướng, Firebase Authentication để xác thực tài khoản, Firebase Realtime Database để lưu trữ dữ liệu và AsyncStorage để duy trì trạng thái đăng nhập trên thiết bị.

Qua cấu trúc source code có thể xác định rằng ứng dụng không triển khai đầy đủ MVC, MVP, MVVM hay Clean Architecture. Project không có các lớp ViewModel, Repository hoặc UseCase độc lập. Thay vào đó, kiến trúc hiện tại được tổ chức theo hướng component-based của React Native, kết hợp React Context để quản lý state và một phần business logic dùng chung, trong khi các màn hình hoặc component có thể truy cập Firebase trực tiếp khi cần.

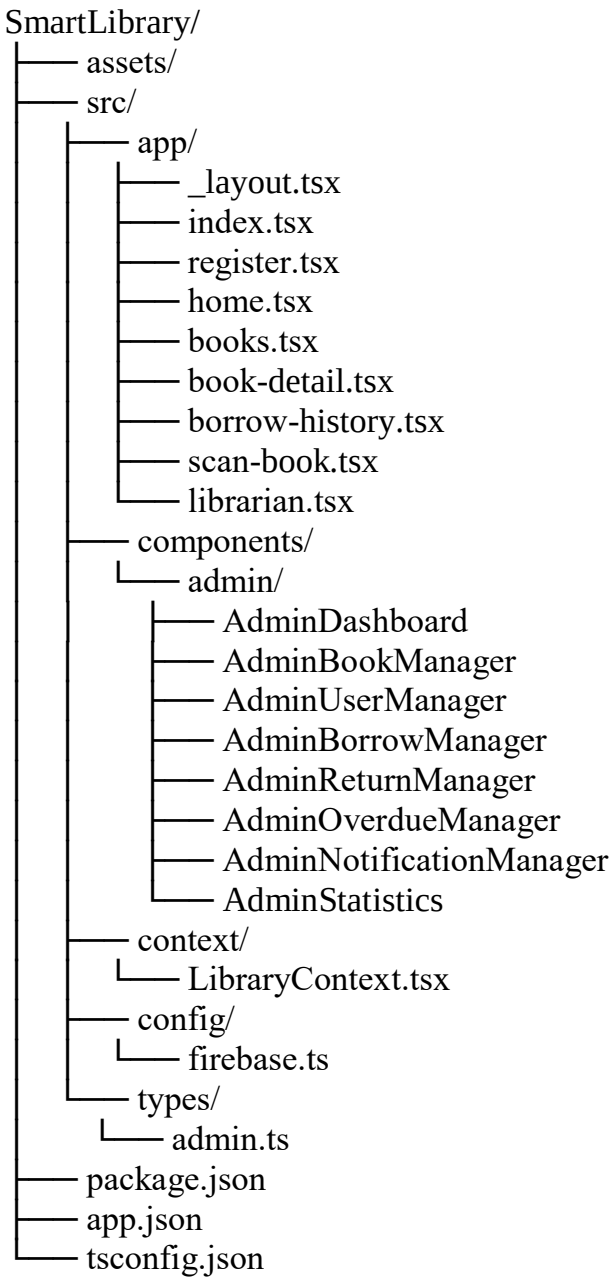
Do đó, cách mô tả phù hợp nhất là: SmartLibrary sử dụng kiến trúc component-based kết hợp Context-based state management và Firebase-backed data access. Kiến trúc có sự phân chia tương đối giữa giao diện, state/logic và dữ liệu nhưng chưa tách hoàn toàn thành các lớp độc lập.

Công nghệ và thư viện chính:

- React Native: xây dựng giao diện ứng dụng di động bằng các component như View, Text, FlatList, TextInput, Image, Modal và TouchableOpacity.
- Expo: cung cấp môi trường phát triển, build và truy cập một số API thiết bị.
- TypeScript: bổ sung kiểu dữ liệu tĩnh cho JavaScript, giúp giảm lỗi khi quản lý dữ liệu và component.
- Expo Router: tổ chức navigation theo cấu trúc file trong thư mục src/app.
- Firebase Authentication: xử lý đăng ký, đăng nhập, đăng xuất và duy trì phiên người dùng.
- Firebase Realtime Database: lưu dữ liệu người dùng, sách và lịch sử mượn/trả.
- AsyncStorage: dùng làm persistence cho trạng thái đăng nhập Firebase trên thiết bị.
- expo-camera: sử dụng camera để quét barcode/QR trong chức năng tìm và mượn sách.

3.2 Cấu trúc source code

Phần chính của ứng dụng được tổ chức trong thư mục src. Cấu trúc dưới đây thể hiện các nhóm file có vai trò trực tiếp trong kiến trúc của ứng dụng.



3.2.1 Nhóm màn hình trong src/app

Thư mục src/app chứa các màn hình chính và đồng thời đóng vai trò định nghĩa route do project sử dụng Expo Router. Tên file tương ứng trực tiếp với đường dẫn điều hướng của ứng dụng.

File	Route	Chức năng chính
index.tsx	/	Đăng nhập và xác định quyền truy cập.
register.tsx	/register	Đăng ký tài khoản mới.
home.tsx	/home	Màn hình chính của người dùng.
books.tsx	/books	Hiển thị và tìm kiếm danh sách sách.
book-detail.tsx	/book-detail	Xem chi tiết, mượn hoặc trả sách.
borrow-history.tsx	/borrow-history	Hiển thị lịch sử mượn/trả.
scan-book.tsx	/scan-book	Quét mã sách bằng camera.
librarian.tsx	/librarian	Màn hình quản trị thư viện.

3.2.2 Nhóm context, config và types

Thư mục src/context chứa LibraryContext.tsx. Đây là thành phần trung tâm ở phía người dùng, dùng để quản lý danh sách sách, lịch sử mượn và các thao tác nghiệp vụ dùng chung. Những màn hình cần dữ liệu thư viện có thể truy cập Context thông qua hook useLibrary().

Thư mục src/config chứa firebase.ts. File này khởi tạo Firebase App, Firebase Authentication và Firebase Realtime Database, sau đó export các đối tượng auth và db cho những phần còn lại của ứng dụng sử dụng.

Thư mục src/types chứa các kiểu dữ liệu TypeScript dùng cho khu vực quản trị, chẳng hạn UserAccount, AdminBook, AdminBorrowRecord và AppNotification. Việc tách type giúp các component quản trị sử dụng cùng một cấu trúc dữ liệu.

3.2.3 Nhóm component quản trị

Phần quản trị được tách thành các component chức năng trong src/components/admin. Mỗi component phụ trách một nhóm nghiệp vụ cụ thể, trong khi librarian.tsx đóng vai trò màn hình tổng và chuyển đổi giữa các chức năng quản trị.

- AdminDashboard: tổng hợp thông tin và trạng thái hệ thống.
- AdminBookManager: quản lý thông tin sách, bao gồm thêm, sửa và xóa.
- AdminUserManager: quản lý người dùng.
- AdminBorrowManager và AdminReturnManager: quản lý quá trình mượn và trả.
- AdminOverdueManager: theo dõi các trường hợp quá hạn.
- AdminNotificationManager: quản lý thông báo.
- AdminStatistics: hiển thị thông tin thống kê.

3.3 Phân tích các thành phần kiến trúc chính

3.3.1 Presentation Layer

Presentation Layer được hình thành từ các screen trong src/app và các component trong src/components. Khác với Android Native, project không sử dụng Activity, Fragment, XML Layout hoặc Jetpack Compose. Giao diện được xây dựng bằng functional component của React Native.

Các screen tiếp nhận thao tác của người dùng, hiển thị dữ liệu và điều hướng sang màn hình khác. Chẳng hạn BooksScreen sử dụng FlatList để hiển thị danh sách sách và TextInput để nhập từ khóa tìm kiếm; BookDetailScreen hiển thị thông tin chi tiết và tiếp nhận thao tác mượn/trả; ScanBookScreen truy cập camera để nhận barcode/QR.

3.3.2 State và Business Logic

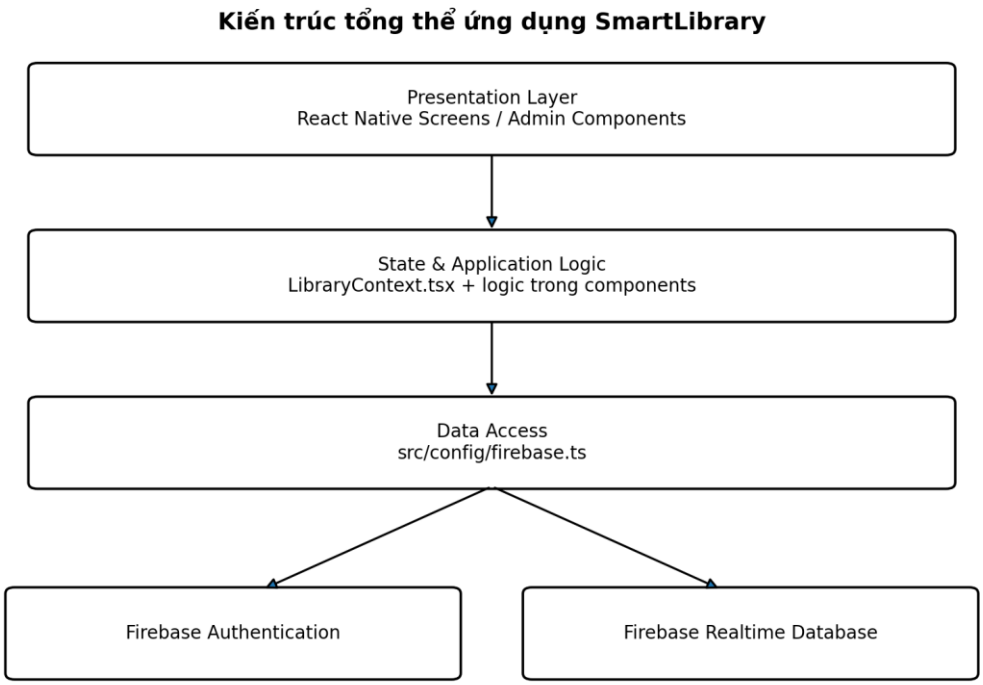
SmartLibrary chưa tách riêng một Business Layer độc lập. Phần logic dùng chung chủ yếu nằm trong LibraryContext.tsx. Context quản lý các state như books, borrowRecords và loading, đồng thời cung cấp các hàm borrowBook(), returnBook() và getBorrowedQuantity().

Một phần logic vẫn nằm trực tiếp trong screen hoặc component. Ví dụ BookDetailScreen kiểm tra số lượng mượn trước khi gọi borrowBook(), còn AdminBookManager vừa quản lý form, kiểm tra dữ liệu, sinh barcode và gọi Firebase để thêm, sửa hoặc xóa sách. Điều này cho thấy Presentation Layer và Business Logic vẫn còn liên kết tương đối chặt.

3.3.3 Data Access Layer

Lớp truy cập dữ liệu của project được triển khai ở mức đơn giản thông qua Firebase SDK. File firebase.ts tạo kết nối và cung cấp các đối tượng auth và db. Project không có Repository hoặc API Service riêng.

Các chức năng phía người dùng thường truy cập dữ liệu thông qua LibraryContext, trong khi một số component quản trị có thể gọi Firebase Realtime Database trực tiếp. Vì vậy luồng truy cập dữ liệu không hoàn toàn thống nhất giữa hai nhóm chức năng.

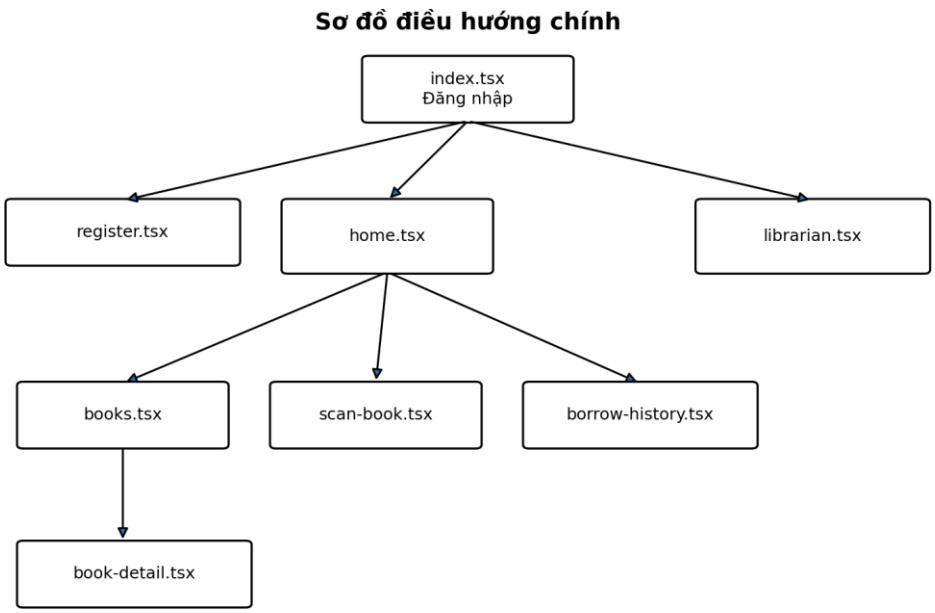


Hình 3.1. Kiến trúc tổng thể ứng dụng SmartLibrary

3.4 Cơ chế điều hướng giữa các màn hình

Expo Router chịu trách nhiệm quản lý navigation. File _layout.tsx khai báo Stack và xác định các route chính. Khi ứng dụng khởi động, trạng thái đăng nhập được kiểm tra; sau đó hệ thống điều hướng người dùng theo role tương ứng.

Đối với tài khoản người dùng thông thường, màn hình home là điểm truy cập chính tới danh sách sách, quét mã và lịch sử mượn. Khi chọn một cuốn sách trong danh sách, id của sách được truyền sang book-detail thông qua router. Đối với tài khoản librarian, hệ thống chuyển trực tiếp sang khu vực quản trị.



Hình 3.2. Sơ đồ điều hướng chính của ứng dụng

3.5 Phân tích luồng xử lý các chức năng chính

3.5.1 Luồng đăng nhập và phân quyền

Người dùng nhập email và mật khẩu tại index.tsx. Screen gọi signInWithEmailAndPassword() của Firebase Authentication. Khi xác thực thành công, ứng dụng sử dụng uid để đọc thông tin tại node users/{uid} của Realtime Database. Trường role được sử dụng để xác định màn hình đích.

Người dùng
↓
index.tsx
↓ signInWithEmailAndPassword()
Firebase Authentication
↓ uid
Firebase Realtime Database: users/{uid}
↓ role
user → /home
librarian → /librarian

3.5.2 Luồng đăng ký tài khoản

Tại register.tsx, tài khoản được tạo bằng createUserWithEmailAndPassword(). Sau đó ứng dụng cập nhật profile cần thiết và ghi thông tin người dùng vào Realtime Database. Như vậy thông tin xác thực được quản lý bởi Firebase Authentication, còn dữ liệu hồ sơ ứng dụng được lưu riêng trong Realtime Database.

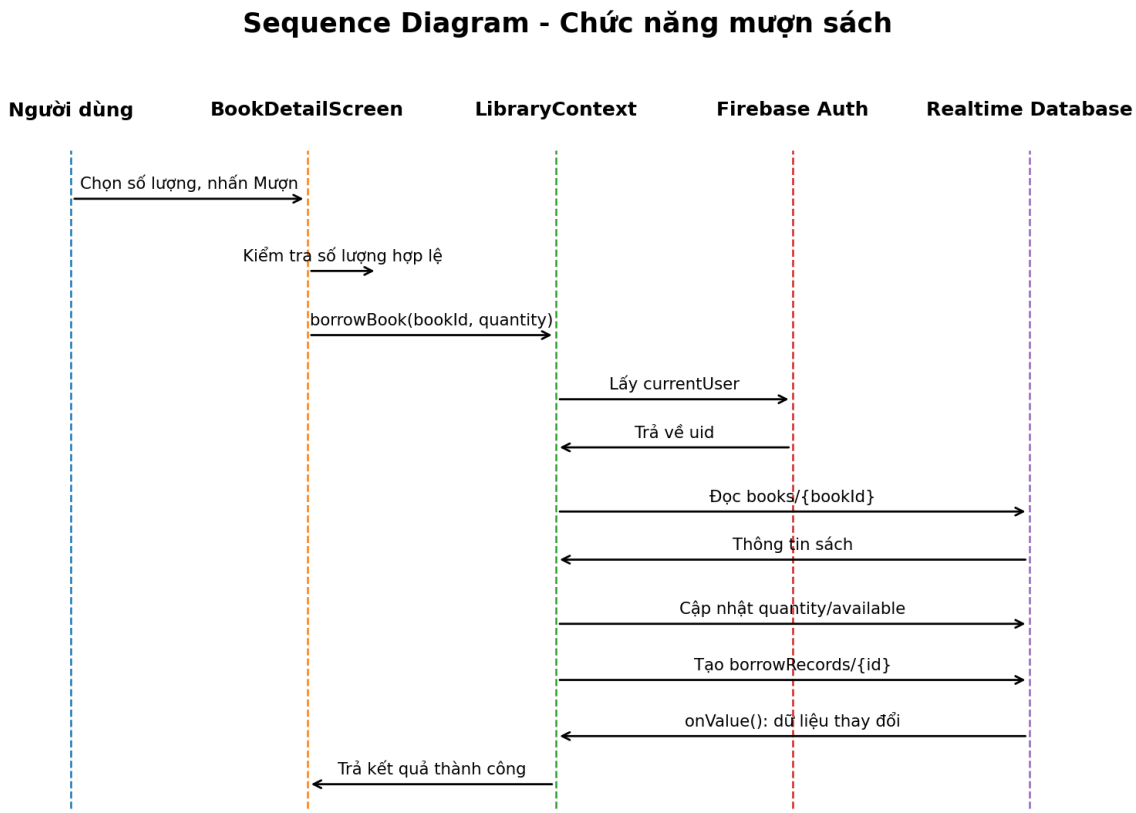
3.5.3 Luồng tìm kiếm và xem chi tiết sách

BooksScreen lấy danh sách sách từ LibraryContext. Khi người dùng nhập từ khóa, việc lọc được thực hiện cục bộ trên mảng books, dựa trên các thuộc tính như tiêu đề, tác giả, thể loại, ISBN hoặc vị trí kệ. Khi chọn một phần tử, router chuyển sang book-detail kèm id sách. BookDetailScreen nhận id bằng useLocalSearchParams() rồi tìm đối tượng tương ứng trong state hiện có.

3.5.4 Luồng mượn sách

Khi người dùng chọn số lượng và nhấn nút mượn tại BookDetailScreen, screen kiểm tra tính hợp lệ của số lượng. Sau đó hàm borrowBook(bookId, quantity) trong LibraryContext được gọi. Context lấy người dùng hiện tại, kiểm tra dữ liệu sách, cập nhật số lượng còn lại và tạo bản ghi mượn mới trong Realtime Database.

Do LibraryContext đăng ký listener onValue(), thay đổi tại Firebase được phản ánh trở lại state của ứng dụng. React sau đó render lại giao diện dựa trên dữ liệu mới. Đây là luồng dữ liệu hai chiều theo cơ chế realtime giữa app và Firebase.



Hình 3.3. Sequence Diagram chức năng mượn sách

3.5.5 Luồng quét mã sách

ScanBookScreen sử dụng CameraView và quyền camera của expo-camera. Khi mã được nhận, hàm xử lý quét so sánh dữ liệu barcode/QR với danh sách books đã có trong LibraryContext. Nếu tìm thấy sách, thông tin được hiển thị để người dùng tiếp tục thao tác mượn. Điểm đáng chú ý là mã quét không bắt buộc phải gửi lên một server tìm kiếm riêng; việc đối chiếu được thực hiện với state đang được đồng bộ từ Firebase.

3.6 Giao tiếp giữa ứng dụng và Firebase

SmartLibrary không thể hiện một backend REST API trung gian trong source mobile hiện tại. App kết nối trực tiếp tới Firebase thông qua JavaScript SDK. Mô hình giao tiếp

này giúp giảm số lượng thành phần phải triển khai trong đồ án nhưng cũng khiến code phía client phụ thuộc trực tiếp vào cấu trúc dữ liệu Firebase.

Firebase Authentication chịu trách nhiệm cho các thao tác như createUserWithEmailAndPassword(), signInWithEmailAndPassword(), onAuthStateChanged(), signOut(), sendPasswordResetEmail() và updateProfile(). Trạng thái xác thực được lưu bền vững bằng AsyncStorage thông qua getReactNativePersistence().

Firebase Realtime Database được sử dụng cho các node dữ liệu chính như users, books và borrowRecords. Các thao tác đọc/ghi sử dụng ref(), get(), set(), update(), push(), remove() hoặc onValue() tùy chức năng. onValue() cho phép ứng dụng nhận thay đổi realtime và cập nhật giao diện mà không cần người dùng tải lại thủ công.

Thành phần	Vai trò	Ví dụ dữ liệu/chức năng
Firebase Authentication	Xác thực và duy trì phiên đăng nhập	Email/password, uid, session
Realtime Database	Lưu dữ liệu nghiệp vụ của hệ thống	users, books, borrowRecords
AsyncStorage	Lưu persistence của phiên trên thiết bị	Trạng thái đăng nhập Firebase
LibraryContext	Đồng bộ dữ liệu và cung cấp logic dùng chung	books, borrowRecords, borrowBook(), returnBook()

3.7 Đánh giá kiến trúc hiện tại

Cách tổ chức hiện tại phù hợp với một ứng dụng đồ án có quy mô vừa. Việc sử dụng Expo Router giúp route được tổ chức rõ ràng theo file; LibraryContext giúp nhiều screen dùng chung dữ liệu sách và lịch sử mượn; các component quản trị được tách riêng theo chức năng nên dễ xác định trách nhiệm ở mức giao diện.

Tuy nhiên, ranh giới giữa Presentation, Business Logic và Data Access chưa hoàn toàn rõ. Một số screen hoặc component thực hiện đồng thời nhiều trách nhiệm. Ví dụ

AdminBookManager có thể vừa xử lý giao diện, validation, dữ liệu form, sinh barcode và thao tác trực tiếp với Firebase. Tương tự, màn hình đăng nhập vừa tiếp nhận dữ liệu người dùng, gọi Authentication, đọc role từ Database và quyết định navigation.

Project cũng chưa có Repository hoặc Service Layer thống nhất. Điều này chưa gây trở ngại lớn ở quy mô hiện tại nhưng khi số lượng chức năng tăng, code có thể khó kiểm thử và bảo trì hơn. Nếu tiếp tục phát triển, có thể tách các thao tác Firebase vào service/repository riêng và giữ screen chủ yếu cho nhiệm vụ hiển thị và xử lý tương tác.

Tóm lại, kiến trúc SmartLibrary hiện tại có thể được xác định là kiến trúc component-based của React Native, sử dụng Expo Router cho navigation, React Context cho state và một phần business logic, đồng thời sử dụng Firebase Authentication và Firebase Realtime Database làm nền tảng dữ liệu. Cách mô tả này phản ánh đúng cấu trúc source hiện có hơn so với việc gán project vào MVVM hoặc Clean Architecture hoàn chỉnh.

3.8 Kết luận chương

Chương 3 đã phân tích cấu trúc và kiến trúc của ứng dụng di động SmartLibrary từ góc nhìn source code. Ứng dụng được xây dựng bằng React Native, Expo và TypeScript; sử dụng Expo Router để điều hướng; LibraryContext để quản lý state và logic dùng chung; Firebase Authentication và Realtime Database để cung cấp chức năng xác thực và lưu trữ dữ liệu. Các luồng đăng nhập, đăng ký, tìm kiếm sách, mượn sách và quét mã đều được tổ chức xoay quanh các thành phần này.

Phân tích cũng cho thấy project đã có sự phân chia chức năng tương đối rõ nhưng chưa tách hoàn toàn thành các lớp độc lập theo mô hình MVVM hoặc Clean Architecture. Đây là cơ sở để mô tả đúng hệ thống hiện tại và đồng thời định hướng việc cải tiến cấu trúc nếu ứng dụng tiếp tục được mở rộng.

CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ

4.1 Mục tiêu và phạm vi đánh giá

Chương này phân tích kết quả của dự án Smart Library – ứng dụng quản lý thư viện được xây dựng bằng React Native, Expo và TypeScript, sử dụng Expo Router để điều hướng, Firebase Authentication để xác thực tài khoản và Firebase Realtime Database để lưu trữ, đồng bộ dữ liệu. Nội dung đánh giá bám sát cấu trúc source code và các luồng nghiệp vụ đã mô tả ở Chương 3, tập trung vào đăng nhập, đăng ký, tìm kiếm sách, xem chi tiết, mượn/trả sách, quét mã và khu vực quản trị.

Cần phân biệt rõ kết quả phân tích từ thiết kế/source code với kết quả thực nghiệm đo được. Tài liệu đầu vào chưa cung cấp nhật ký kiểm thử, số lượng ca chạy thực tế, cấu hình thiết bị, thời gian phản hồi đo bằng công cụ hoặc dữ liệu vận hành phần cứng ESP32. Vì vậy, chương này không tự tạo số liệu phần trăm thành công, độ trễ hay độ chính xác. Các nhận định về hiệu năng được trình bày theo hướng định tính dựa trên kiến trúc hiện có; bảng kiểm thử là bộ ca kiểm thử đề xuất để nhóm thực hiện và điền kết quả thực tế trước khi nộp báo cáo.

4.1.1 Mục tiêu đánh giá

- Đối chiếu các chức năng đã được mô tả trong source code với mục tiêu ban đầu của dự án.
- Phân tích tính đúng đắn của những luồng nghiệp vụ cốt lõi, đặc biệt là xác thực, phân quyền, tìm kiếm và mượn/trả sách.
- Đánh giá cách tổ chức dữ liệu, cơ chế đồng bộ từ Firebase và mức độ phản hồi của giao diện theo đặc điểm kiến trúc.
- Nhận diện các giới hạn kỹ thuật có thể ảnh hưởng đến độ tin cậy, khả năng bảo trì và khả năng mở rộng.

- Đề xuất quy trình kiểm thử định lượng để có thể xác nhận chất lượng hệ thống bằng dữ liệu thực nghiệm.

4.1.2 Cơ sở và phương pháp đánh giá

Phương pháp đánh giá được chia thành ba lớp:

- Thứ nhất là phân tích tĩnh: xem xét vai trò của các màn hình, LibraryContext, cấu hình Firebase và các hàm nghiệp vụ được nêu trong Chương 3.
- Thứ hai là phân tích luồng: lần theo đường đi của dữ liệu từ thao tác người dùng đến Firebase và ngược lại.
- Thứ ba là kiểm thử chức năng đề xuất: xác định đầu vào, thao tác, kết quả mong đợi và điều kiện biên để nhóm có thể thực hiện trên bản chạy thực tế.

Lớp đánh giá	Nội dung	Giá trị sử dụng	Giới hạn
Phân tích tĩnh	Cấu trúc src/app, LibraryContext.tsx, firebase.ts, component quản trị	Nhận diện trách nhiệm từng thành phần và điểm phụ thuộc	Không chứng minh được hành vi runtime trong mọi tình huống
Phân tích luồng	Đăng nhập, đăng ký, tìm kiếm, mượn/trả, quét mã	Xác định các bước xử lý và điểm có thể phát sinh lỗi	Dựa trên luồng được mô tả trong tài liệu
Kiểm thử chức năng	Ca kiểm thử bình thường, dữ liệu sai, mất kết nối, phân quyền	Có thể xác nhận trực tiếp trên bản chạy	Cần chạy thực tế và ghi nhận kết quả
Đánh giá hiệu năng	Thời gian phản hồi, độ trễ đồng bộ, độ ổn định	Định lượng trải nghiệm và tải hệ thống	Chưa có số đo thực tế trong tài liệu đầu vào

4.2 Kết quả triển khai các chức năng chính

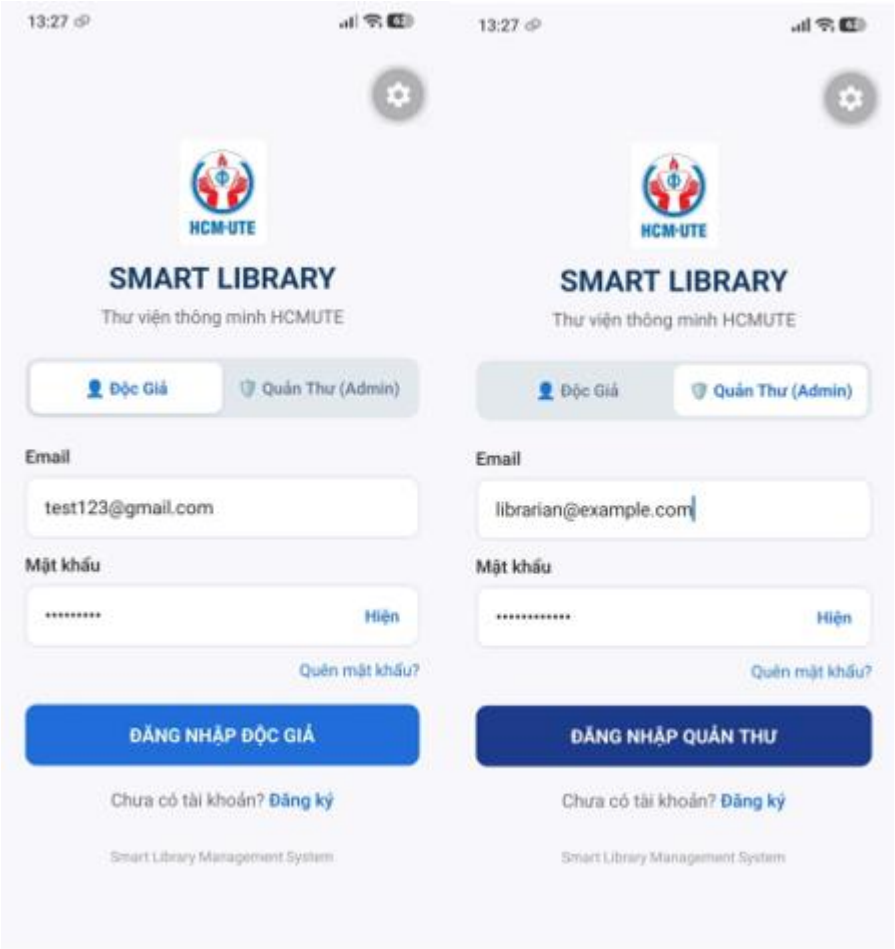
Sau quá trình thiết kế và phát triển, ứng dụng Smart Library đã xây dựng được các nhóm chức năng phục vụ hai đối tượng sử dụng chính là độc giả và quản thư. Các chức

năng được tổ chức thành những màn hình riêng biệt, bao gồm đăng nhập, đăng ký tài khoản, tìm kiếm và mượn sách, theo dõi lịch sử mượn/trả, quét mã sách và quản lý thư viện.

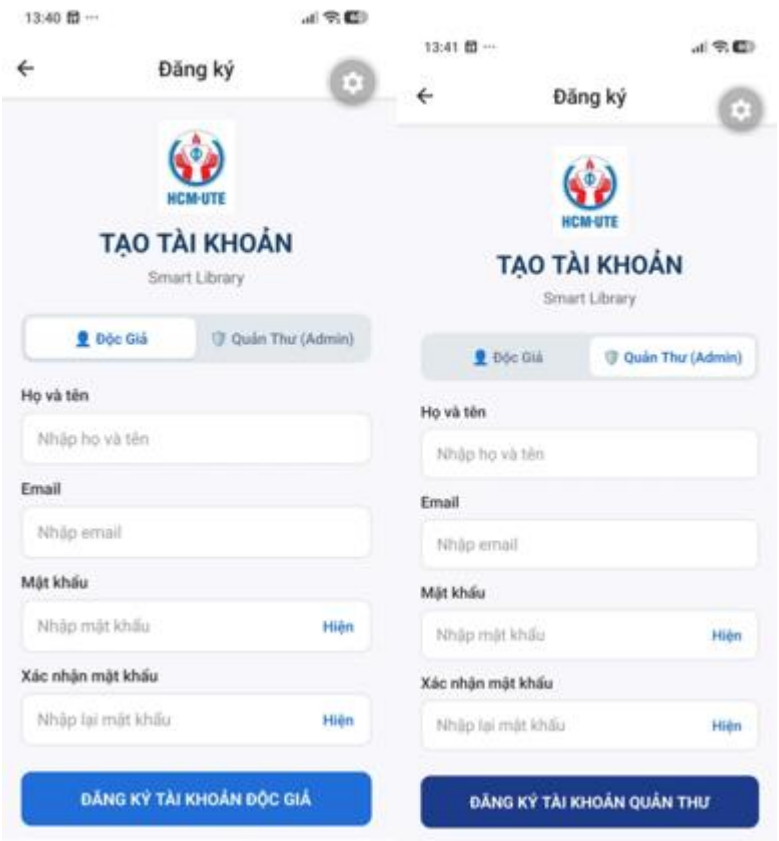
Ứng dụng được phát triển bằng React Native kết hợp Expo và TypeScript, sử dụng Firebase Authentication để xác thực tài khoản và Firebase Realtime Database để lưu trữ, đồng bộ dữ liệu. Kết quả triển khai được thể hiện qua các ảnh chụp giao diện thực tế và được đối chiếu với các luồng xử lý nghiệp vụ đã phân tích ở Chương 3.

Phần đánh giá tập trung vào khả năng tổ chức giao diện, mức độ đáp ứng chức năng, cách hiển thị dữ liệu và sự liên kết giữa các thành phần trong hệ thống. Những số liệu xuất hiện trên giao diện được sử dụng để minh họa trạng thái hoạt động tại thời điểm chụp, không được xem là kết quả đo hiệu năng hoặc kiểm thử định lượng.

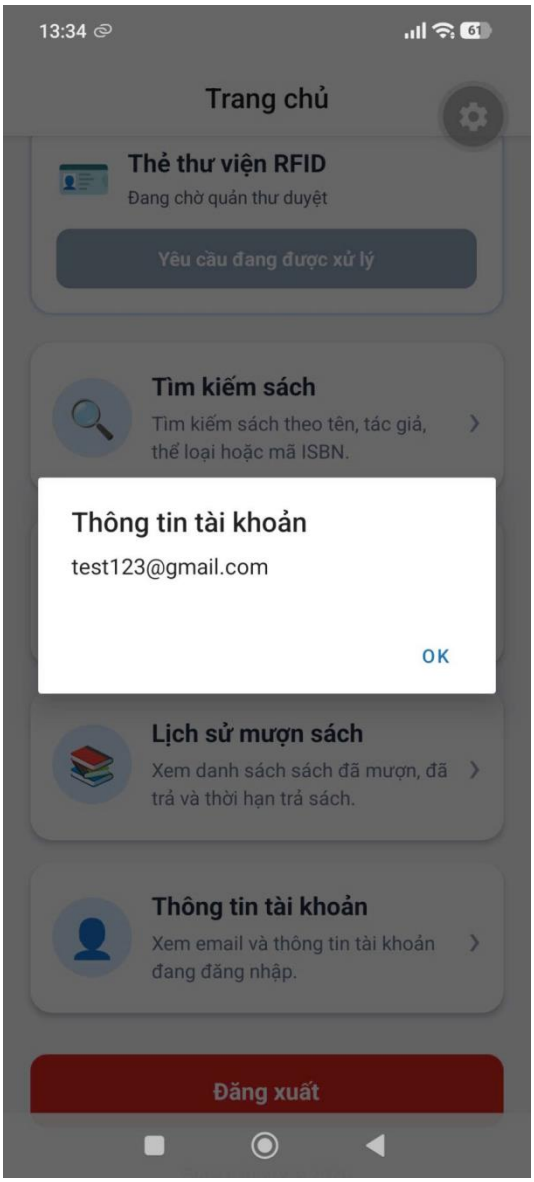
4.2.1 Đăng nhập, đăng ký và phân quyền



Hình 4.3.1. Giao diện đăng nhập của độc giả và quản thư



Hình 4.3.2. Giao diện đăng ký của ứng dụng



Hình 4.3.3. Thông tin tài khoản email của ứng dụng

Chức năng đăng nhập và đăng ký tài khoản là thành phần đầu tiên trong quá trình sử dụng ứng dụng Smart Library. Hệ thống được xây dựng với hai nhóm người dùng là độc giả (User) và quản thư (Librarian), mỗi nhóm có giao diện và phạm vi chức năng riêng, phù hợp với vai trò trong hoạt động quản lý thư viện.

Qua hình ảnh giao diện thực tế, màn hình đăng nhập được thiết kế thống nhất về bố cục, màu sắc và nhận diện ứng dụng. Người dùng có thể lựa chọn vai trò độc giả hoặc quản thư, nhập địa chỉ email và mật khẩu để thực hiện đăng nhập. Giao diện đồng thời cung cấp

tùy chọn hiển thị mật khẩu, khôi phục mật khẩu và chuyển sang màn hình đăng ký tài khoản mới. Các thành phần được bố trí rõ ràng, giúp người dùng xác định được thông tin cần nhập và thao tác cần thực hiện.

Đối với chức năng đăng ký, ứng dụng cung cấp biểu mẫu gồm họ và tên, địa chỉ email, mật khẩu và xác nhận mật khẩu. Việc bổ sung trường xác nhận mật khẩu góp phần hạn chế sai sót trong quá trình tạo tài khoản. Hai biểu mẫu đăng ký cho độc giả và quản thư có cấu trúc tương đồng, tạo sự nhất quán trong trải nghiệm sử dụng.

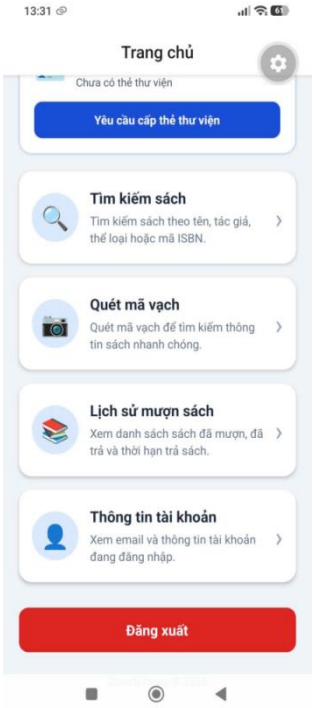
Về cơ chế xử lý, ứng dụng sử dụng Firebase Authentication để xác thực thông qua email và mật khẩu. Sau khi xác thực thành công, hệ thống truy xuất thông tin tài khoản tại node users/{uid} trong Firebase Realtime Database nhằm xác định vai trò tương ứng. Tài khoản độc giả được điều hướng đến màn hình /home, trong khi tài khoản quản thư được chuyển đến /librarian.

Kết quả thể hiện qua các ảnh chụp cho thấy ứng dụng đã xây dựng được giao diện đăng nhập theo vai trò và màn hình chính tương ứng cho từng nhóm người dùng. Hình ảnh hộp thoại thông tin tài khoản cũng minh họa việc ứng dụng có thể hiển thị địa chỉ email của tài khoản đang đăng nhập. Đây là cơ sở cho những chức năng yêu cầu xác định danh tính người sử dụng, đặc biệt là quản lý lịch sử mượn sách và các nghiệp vụ dành riêng cho quản thư.

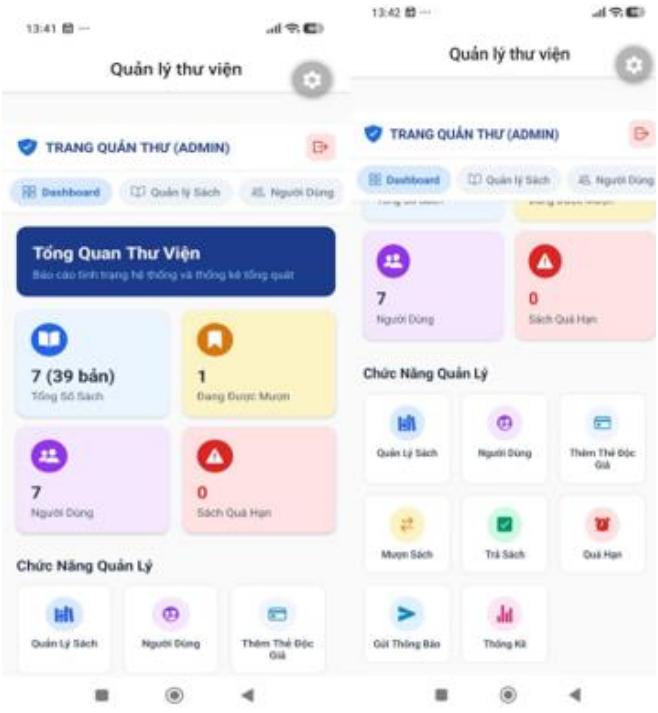
Việc kết hợp Firebase Authentication với Realtime Database giúp phân tách thông tin xác thực và dữ liệu nghiệp vụ, giảm khối lượng xử lý đăng nhập phải tự xây dựng. Tuy nhiên, lựa chọn vai trò trên giao diện chỉ là một phần của cơ chế phân quyền. Do ứng dụng có hiển thị tùy chọn đăng ký tài khoản quản thư, hệ thống cần kiểm soát việc cấp quyền quản trị bằng các quy tắc bảo mật phù hợp, tránh trường hợp người dùng thông thường tự cấp quyền truy cập dữ liệu quản lý.

Nhìn chung, nhóm chức năng đăng nhập và đăng ký đã hình thành được luồng truy cập cơ bản cho hai nhóm người dùng, tạo nền tảng để triển khai các chức năng nghiệp vụ tiếp theo.

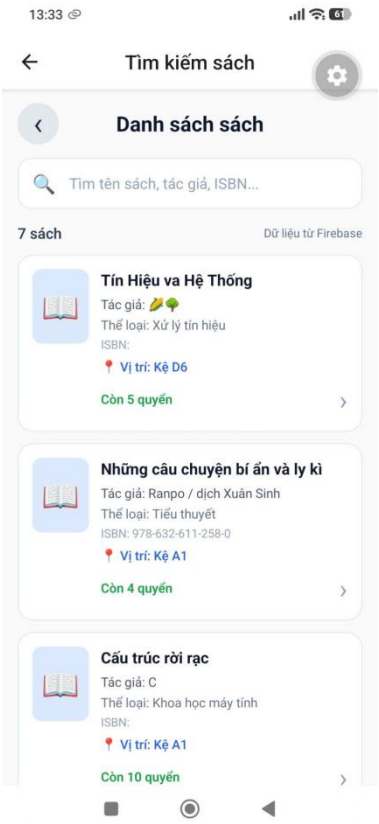
4.2.2 Tìm kiếm sách và xem chi tiết



Hình 4.3.4. Giao diện chính của ứng dụng độc giả



Hình 4.3.5. Giao diện chính ứng dụng của bên quản thư



Hình 4.3.6. Danh sách sách của thư viện

Chức năng tìm kiếm sách được xây dựng nhằm hỗ trợ độc giả tiếp cận danh mục tài liệu của thư viện mà không cần tra cứu thủ công. Đây là một trong những chức năng quan trọng nhất của ứng dụng, đóng vai trò kết nối giữa người sử dụng và nguồn tài nguyên sách đang được quản lý.

Tại giao diện chính của độc giả, các chức năng được tổ chức theo từng mục riêng biệt, bao gồm đăng ký thẻ thư viện, tìm kiếm sách, quét mã vạch, lịch sử mượn sách và thông tin tài khoản. Cách bố trí theo các thẻ chức năng giúp người dùng dễ dàng quan sát và lựa chọn thao tác phù hợp với nhu cầu. Giao diện tương đối đơn giản, hạn chế việc phải chuyển qua nhiều cấp menu để truy cập các tính năng thường xuyên sử dụng.

Đối với màn hình tìm kiếm, ứng dụng hiển thị danh sách sách dưới dạng các thẻ thông tin. Mỗi mục sách cung cấp những thuộc tính cần thiết như tên sách, tác giả, thể loại, vị trí lưu trữ và số lượng còn lại. Trong ảnh chụp thực tế, danh sách đang hiển thị 7 đầu sách, trong đó có các tài liệu như “Tín Hiệu và Hệ Thống”, “Những câu chuyện bí ẩn và ly

kỳ” và “Cấu trúc rời rạc”. Việc hiển thị số lượng sách còn lại ngay trong danh sách giúp độc giả bước đầu xác định khả năng mượn tài liệu trước khi chuyển đến màn hình chi tiết.

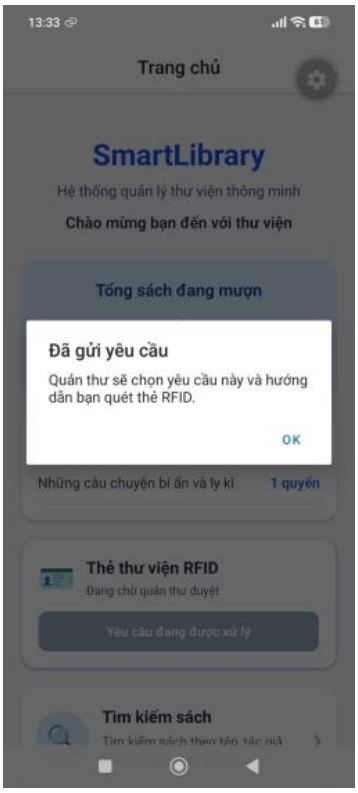
Theo cấu trúc chương trình, màn hình books.tsx lấy dữ liệu sách từ LibraryContext và thực hiện lọc trên danh sách đã được tải về ứng dụng. Người dùng có thể tìm kiếm dựa trên các thuộc tính như tên sách, tác giả, thể loại, ISBN hoặc vị trí kệ sách. Khi lựa chọn một đầu sách, ứng dụng sử dụng mã định danh để chuyển đến màn hình chi tiết, từ đó truy xuất thông tin của tài liệu tương ứng.

Cách tổ chức này có ưu điểm là đơn giản hóa quá trình tìm kiếm đối với danh mục có quy mô nhỏ hoặc vừa. Khi dữ liệu đã được tải vào bộ nhớ ứng dụng, thao tác lọc có thể được xử lý trực tiếp mà không phải gửi một yêu cầu mạng mới cho từng ký tự nhập vào. Đồng thời, việc sử dụng dữ liệu chung từ LibraryContext giúp màn hình tìm kiếm và màn hình chi tiết sử dụng cùng nguồn thông tin.

Về kết quả triển khai, các ảnh thực tế xác nhận giao diện danh mục sách đã có khả năng trình bày nhiều đầu sách cùng thông tin tồn kho và vị trí lưu trữ. Luồng xem chi tiết cũng đã được tổ chức trong mã nguồn; tuy nhiên, nhóm hình minh họa hiện tại chưa thể hiện riêng toàn bộ giao diện chi tiết sách để đánh giá đầy đủ các trường thông tin được hiển thị.

Nhìn chung, chức năng tìm kiếm đã đáp ứng được nhu cầu cơ bản về tra cứu tài liệu trong thư viện. Khi số lượng sách tăng lên, nhóm có thể tiếp tục tối ưu khả năng lọc, xử lý từ khóa tiếng Việt và phương thức tải dữ liệu nhằm duy trì trải nghiệm sử dụng ổn định.

4.2.3 Mượn sách và cập nhật số lượng



Hình 4.3.7. Yêu cầu mượn sách được thực hiện

Chức năng mượn sách là nghiệp vụ cốt lõi của Smart Library, cho phép chuyển hoạt động đăng ký mượn tài liệu từ phương thức thủ công sang thao tác trên ứng dụng di động. Chức năng này có mối liên hệ trực tiếp với thông tin người dùng, danh mục sách, số lượng tồn kho và lịch sử giao dịch.

Theo luồng xử lý đã xây dựng, người dùng truy cập thông tin sách, lựa chọn số lượng cần mượn và thực hiện yêu cầu mượn. Tại màn hình chi tiết, hệ thống kiểm tra dữ liệu đầu vào trước khi gọi hàm borrowBook(bookId, quantity) được quản lý trong LibraryContext. Hàm nghiệp vụ sử dụng thông tin tài khoản hiện tại và dữ liệu sách để xử lý yêu cầu, cập nhật số lượng còn lại và tạo bản ghi mượn trong Firebase Realtime Database.

Hình ảnh thực tế minh họa ứng dụng có phản hồi bằng hộp thoại khi thực hiện yêu cầu mượn sách. Bên cạnh đó, các ảnh chụp tại màn hình lịch sử của độc giả và khu vực quản lý mượn sách của quản thư đều thể hiện giao dịch có trạng thái "Đang mượn". Những thông tin này cho thấy các thành phần giao diện cần thiết để tiếp nhận và theo dõi giao dịch mượn đã được xây dựng.

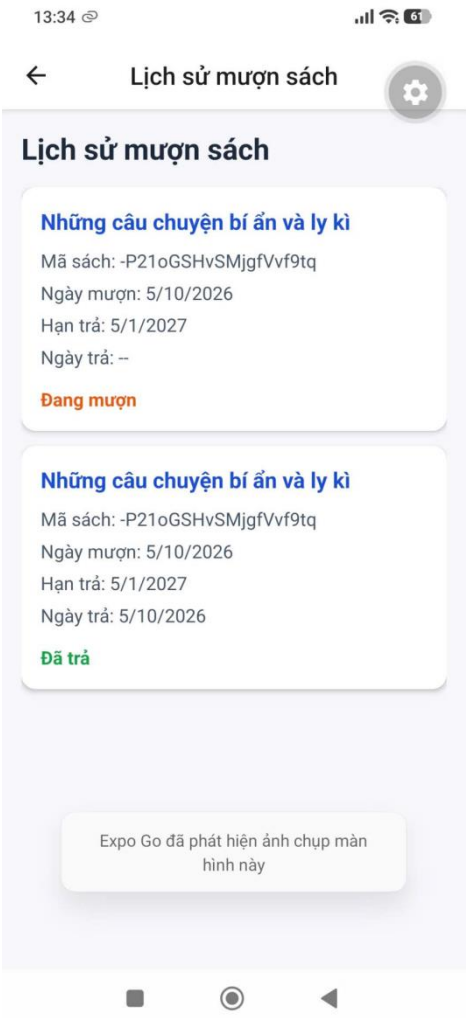
Một điểm đáng chú ý là dữ liệu mượn sách được sử dụng ở cả hai phía người dùng. Độc giả có thể xem trạng thái tài liệu đã đăng ký mượn, trong khi quản thư có giao diện theo dõi những yêu cầu và giao dịch đang hoạt động. Sự phân chia này hỗ trợ quá trình giám sát và xử lý nghiệp vụ, đồng thời hạn chế việc theo dõi giao dịch bằng các danh sách độc lập.

Về mặt dữ liệu, ứng dụng sử dụng Firebase Realtime Database kết hợp cơ chế lắng nghe thay đổi onValue() trong LibraryContext. Khi dữ liệu sách hoặc bản ghi mượn được cập nhật trên Firebase, những màn hình sử dụng nguồn dữ liệu tương ứng có thể nhận trạng thái mới và hiển thị lại thông tin mà không cần người dùng tải lại thủ công.

Kết quả triển khai cho thấy hệ thống đã có các thành phần chính phục vụ việc tạo yêu cầu mượn, quản lý trạng thái giao dịch và hiển thị thông tin ở hai vai trò sử dụng. Đây là bước tiến quan trọng trong quá trình số hóa hoạt động lưu thông tài liệu của thư viện.

Tuy nhiên, để bảo đảm tính chính xác khi vận hành thực tế, hệ thống cần kiểm soát các trường hợp mượn vượt số lượng còn lại, nhiều người cùng mượn một đầu sách hoặc một yêu cầu được gửi nhiều lần. Các thao tác thay đổi số lượng tồn kho và tạo bản ghi giao dịch cần được xử lý nhất quán để tránh tình trạng sai lệch dữ liệu. Những nội dung này cần được xác nhận thêm bằng các ca kiểm thử nghiệp vụ, thay vì chỉ dựa trên kết quả hiển thị của giao diện.

4.2.4 Trả sách và lịch sử mượn



Hình 4.3.8. Lịch sử mượn sách

Bên cạnh hoạt động mượn tài liệu, chức năng trả sách và quản lý lịch sử được triển khai nhằm hỗ trợ theo dõi toàn bộ quá trình sử dụng tài nguyên thư viện. Đây là cơ sở để xác định trạng thái sách, kiểm soát tài liệu đang được mượn và cung cấp thông tin phục vụ quản lý.

Giao diện lịch sử mượn sách của độc giả hiển thị các bản ghi theo từng mục riêng biệt. Mỗi bản ghi bao gồm tên sách, mã sách, ngày mượn, hạn trả, ngày trả và trạng thái giao dịch. Trong ảnh chụp thực tế, ứng dụng hiển thị hai trạng thái "Đang mượn" và "Đã trả", được phân biệt bằng màu sắc. Việc thể hiện trực tiếp trạng thái giúp độc giả dễ dàng nhận biết tài liệu nào vẫn đang được sử dụng và tài liệu nào đã hoàn thành quá trình mượn/trả.

Đối với bản ghi đang mượn, trường ngày trả chưa có giá trị, phù hợp với trạng thái giao dịch chưa kết thúc. Trong khi đó, bản ghi đã trả có thông tin ngày trả và trạng thái hoàn thành. Cách trình bày này giúp lịch sử giao dịch phản ánh các giai đoạn khác nhau trong quá trình mượn sách, đồng thời cung cấp cơ sở để độc giả chủ động theo dõi hạn trả tài liệu.

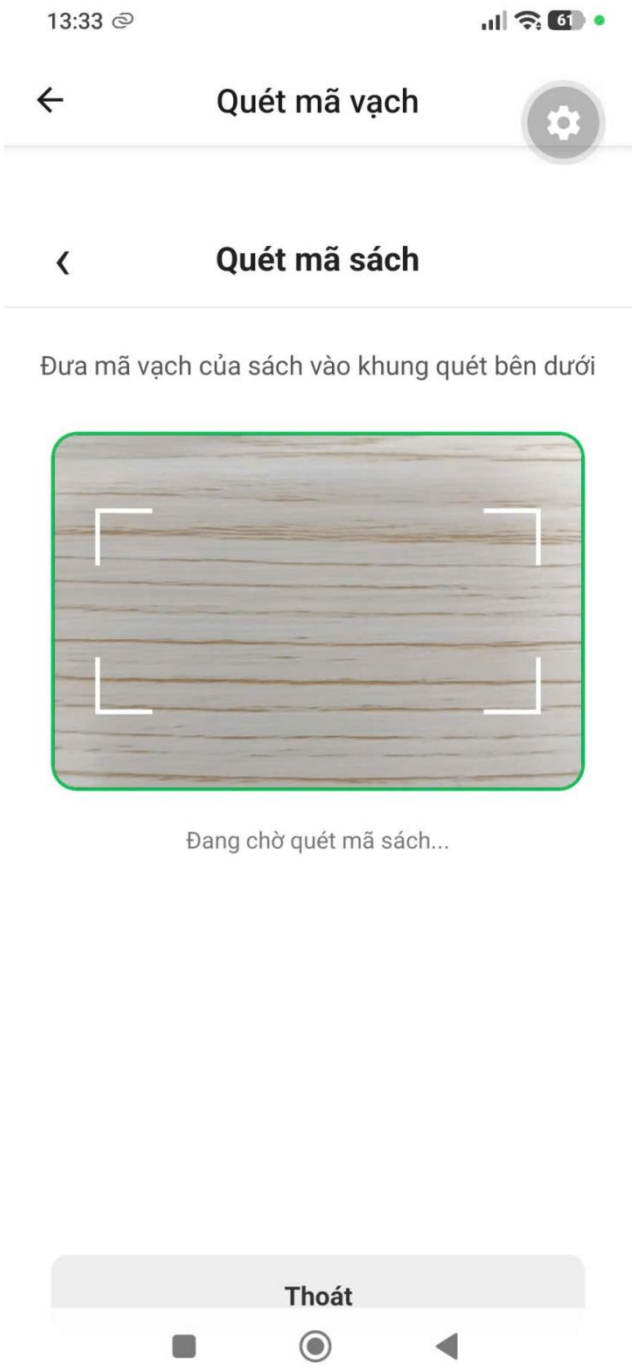
Về phía quản thư, ứng dụng cung cấp màn hình quản lý trả sách, hiển thị danh sách những tài liệu đang được mượn cùng thông tin người mượn, ngày mượn và hạn trả. Trong ảnh thực tế, màn hình này hiển thị một giao dịch đang mượn kèm nút "Xác Nhận Đã Trả Sách", cho phép quản thư thực hiện bước xác nhận hoàn tất việc trả tài liệu.

Theo kiến trúc của hệ thống, nghiệp vụ trả sách được xử lý thông qua hàm returnBook() trong LibraryContext. Về nguyên tắc, quá trình này cần thay đổi trạng thái giao dịch và cập nhật lại số lượng sách hiện có trong cơ sở dữ liệu. Các thay đổi sau đó được sử dụng để làm mới thông tin lịch sử và danh mục sách.

Kết quả thể hiện trên các màn hình cho thấy Smart Library đã xây dựng được giao diện theo dõi lịch sử mượn/trả cho độc giả và giao diện xác nhận trả sách dành cho quản thư. Việc quản lý cùng một loại giao dịch từ hai phía tạo điều kiện thuận lợi cho công tác đối chiếu, giảm sự phụ thuộc vào phương pháp ghi chép thủ công.

Tuy vậy, quy trình trả sách cần bảo đảm rằng một giao dịch chỉ được hoàn tất một lần và số lượng tồn kho chỉ được tăng tương ứng với số lượng sách thực tế đã trả. Đây là yêu cầu cần thiết để duy trì tính toàn vẹn dữ liệu khi ứng dụng được triển khai với nhiều người dùng.

4.2.5 Quét barcode/QR bằng camera



Hình 4.3.9. Giao diện quét mã sách bằng camera

Chức năng quét mã sách được phát triển nhằm hỗ trợ nhận diện và tra cứu tài liệu thông qua camera của thiết bị di động. Đây là phương thức bổ sung cho chức năng tìm kiếm bằng từ khóa, hướng đến việc rút ngắn số thao tác khi người dùng cần truy cập thông tin của một cuốn sách có mã nhận diện.

Giao diện quét mã được thiết kế với vùng hiển thị camera nằm ở trung tâm màn hình, kết hợp khung định vị để hướng dẫn người dùng đặt mã sách vào khu vực nhận diện. Bên trên khung quét là dòng hướng dẫn thao tác, trong khi bên dưới có trạng thái cho biết ứng dụng đang chờ quét mã sách. Giao diện đồng thời cung cấp nút thoát, giúp người dùng chủ động kết thúc quá trình quét.

Qua ảnh chụp thực tế, có thể xác nhận màn hình quét đã được xây dựng và camera có khả năng hiển thị hình ảnh trong khu vực nhận diện. Điều này cho thấy phần giao diện và hoạt động truy cập camera đã được thể hiện trên thiết bị chạy ứng dụng. Tuy nhiên, ảnh hiện tại chỉ ghi nhận trạng thái đang chờ quét, chưa có kết quả minh họa một mã được nhận diện thành công.

Về kỹ thuật, chức năng được triển khai thông qua CameraView của thư viện expo-camera. Khi nhận được dữ liệu barcode hoặc QR, ứng dụng đối chiếu nội dung mã với danh sách sách đã được đồng bộ trong LibraryContext. Nếu tìm được tài liệu có mã tương ứng, ứng dụng có thể cung cấp thông tin sách để người dùng tiếp tục các thao tác liên quan.

Việc tận dụng dữ liệu sách sẵn có trong ứng dụng giúp đơn giản hóa quy trình nhận diện, không yêu cầu xây dựng một dịch vụ tra cứu mã độc lập. Chức năng này cũng tạo nền tảng cho việc ứng dụng mã định danh trong quản lý sách, hỗ trợ phân biệt các tài liệu khi cần tra cứu hoặc xử lý nghiệp vụ.

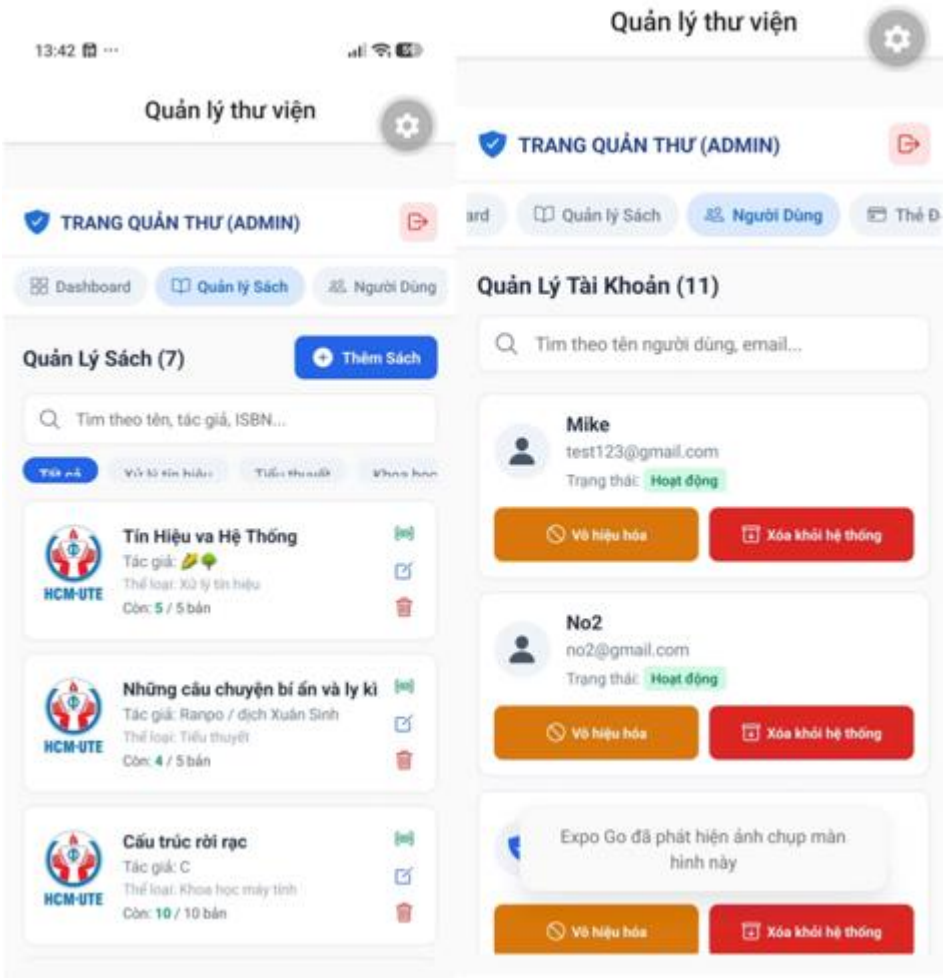
Nhìn chung, kết quả triển khai đã thể hiện được giao diện quét mã và khả năng sử dụng camera của thiết bị. Để đánh giá đầy đủ chức năng nhận diện, cần tiếp tục kiểm tra với mã hợp lệ, mã không tồn tại trong cơ sở dữ liệu và các điều kiện ảnh hưởng đến chất lượng quét như ánh sáng hoặc khoảng cách camera.

4.2.6 Khu vực quản trị thư viện

Khu vực quản trị thư viện là thành phần tập trung các chức năng quản lý của Smart Library, phục vụ đối tượng quản thư trong quá trình điều hành và giám sát hoạt động sử dụng tài nguyên. So với giao diện độc giả, khu vực này có phạm vi chức năng rộng hơn,

bao gồm quản lý sách, người dùng, thẻ độc giả, giao dịch mượn/trả, sách quá hạn, thông báo và thống kê.

Giao diện tổng quan được thiết kế theo dạng bảng điều khiển (Dashboard), giúp quản thư quan sát nhanh những thông tin quan trọng. Tại thời điểm chụp màn hình, Dashboard thể hiện 7 đầu sách với tổng cộng 39 bản sách, 1 giao dịch đang mượn, 7 người dùng và 0 trường hợp sách quá hạn. Đây là những giá trị được hiển thị ở thời điểm ghi nhận, không phải số liệu kiểm thử hiệu năng. Bên dưới các thẻ tổng quan là các lối tắt dẫn đến những nhóm nghiệp vụ quản trị.



Hình 4.3.10. Giao diện quản lý danh mục sách và tài khoản người dùng

a. Quản lý danh mục sách

Chức năng quản lý sách cho phép quản thư theo dõi và thao tác trên danh mục tài liệu. Giao diện hiển thị danh sách sách cùng những thông tin như tên sách, tác giả, thể loại và số lượng bản sách hiện có so với tổng số bản được quản lý.

Trong ảnh thực tế, màn hình quản lý đang hiển thị 7 đầu sách, có thanh tìm kiếm và các tùy chọn lọc danh mục. Mỗi mục sách được bố trí các biểu tượng thao tác phục vụ chỉnh sửa hoặc xóa thông tin. Ngoài ra, giao diện còn cung cấp chức năng thêm sách mới, tạo điều kiện để mở rộng danh mục tài liệu.

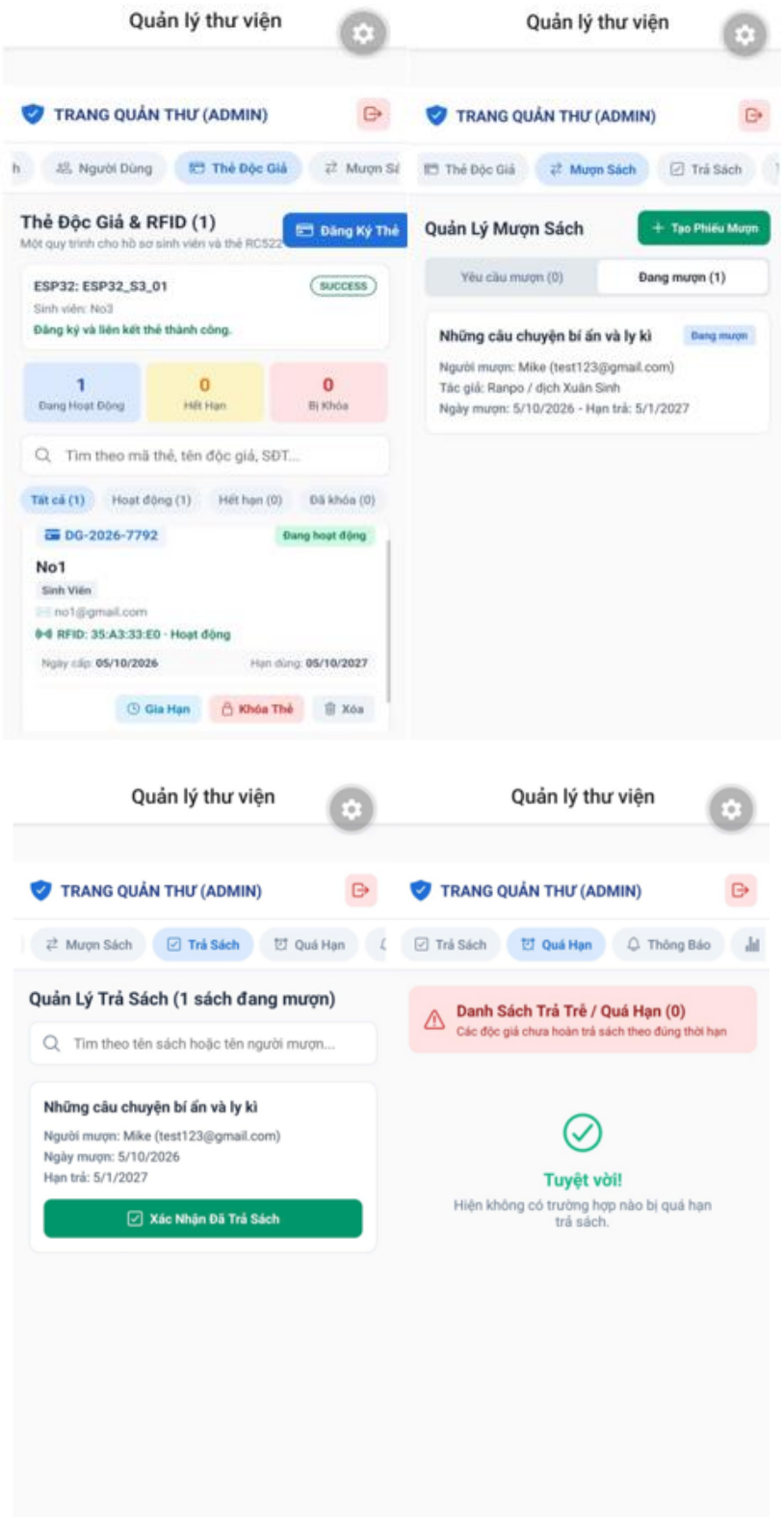
Việc quản lý thông tin sách trên ứng dụng giúp quản thư tập trung dữ liệu về tài liệu và tồn kho tại một nơi. Khi thông tin được cập nhật trong Firebase, các thành phần có sử dụng dữ liệu danh mục có thể tiếp nhận thay đổi thông qua cơ chế đồng bộ đã xây dựng.

b. Quản lý tài khoản người dùng

Màn hình quản lý người dùng cung cấp danh sách các tài khoản có trong hệ thống, bao gồm tên, email và trạng thái hoạt động. Giao diện có thanh tìm kiếm theo thông tin người dùng, hỗ trợ quản thư xác định tài khoản cần quản lý.

Trong ảnh chụp, danh sách hiển thị 11 tài khoản cùng các thao tác vô hiệu hóa hoặc xóa tài khoản. Điều này cho thấy ứng dụng đã tổ chức được khu vực quản lý người dùng riêng biệt, đồng thời cung cấp các chức năng cần thiết để kiểm soát tình trạng sử dụng tài khoản.

Đối với những thao tác có khả năng ảnh hưởng trực tiếp đến quyền truy cập, việc xác nhận trước khi thực hiện và kiểm tra quyền quản trị tại cơ sở dữ liệu là yêu cầu quan trọng. Nhờ đó, hệ thống có thể hạn chế những thay đổi ngoài ý muốn hoặc truy cập không được phép.



Hình 4.3.11. Giao diện quản lý thẻ RFID, mượn sách, trả sách và sách quá hạn

c. Quản lý thẻ độc giả và RFID

Một chức năng đáng chú ý trong khu vực quản trị là quản lý thẻ độc giả kết hợp công nghệ RFID. Giao diện này hỗ trợ lưu trữ và theo dõi thông tin thẻ thư viện, tạo điều kiện liên kết tài khoản độc giả với mã thẻ nhận diện.

Hình ảnh thực tế thể hiện khu vực quản lý thẻ có thông tin thiết bị ESP32, trạng thái đăng ký và liên kết thẻ thành công, cùng danh sách thẻ được quản lý. Tại thời điểm chụp, hệ thống hiển thị 1 thẻ đang hoạt động, 0 thẻ hết hạn và 0 thẻ bị khóa.

Bên cạnh thông tin trạng thái, giao diện cung cấp các thao tác như gia hạn, khóa thẻ và xóa thẻ. Những chức năng này hỗ trợ quản thư xử lý các trường hợp cần thay đổi quyền sử dụng thẻ hoặc cập nhật thông tin của độc giả.

Kết quả trên giao diện cho thấy module quản lý thẻ RFID đã được đưa vào ứng dụng, đồng thời có trạng thái phản hồi cho quá trình đăng ký và liên kết thẻ. Đây là cơ sở để mở rộng hệ thống theo hướng kết hợp ứng dụng di động và thiết bị phần cứng trong quản lý thư viện. Tuy nhiên, cần có kết quả kiểm thử phần cứng riêng để đánh giá độ ổn định và khả năng nhận diện thẻ trong nhiều điều kiện sử dụng.

d. Quản lý mượn, trả sách và sách quá hạn

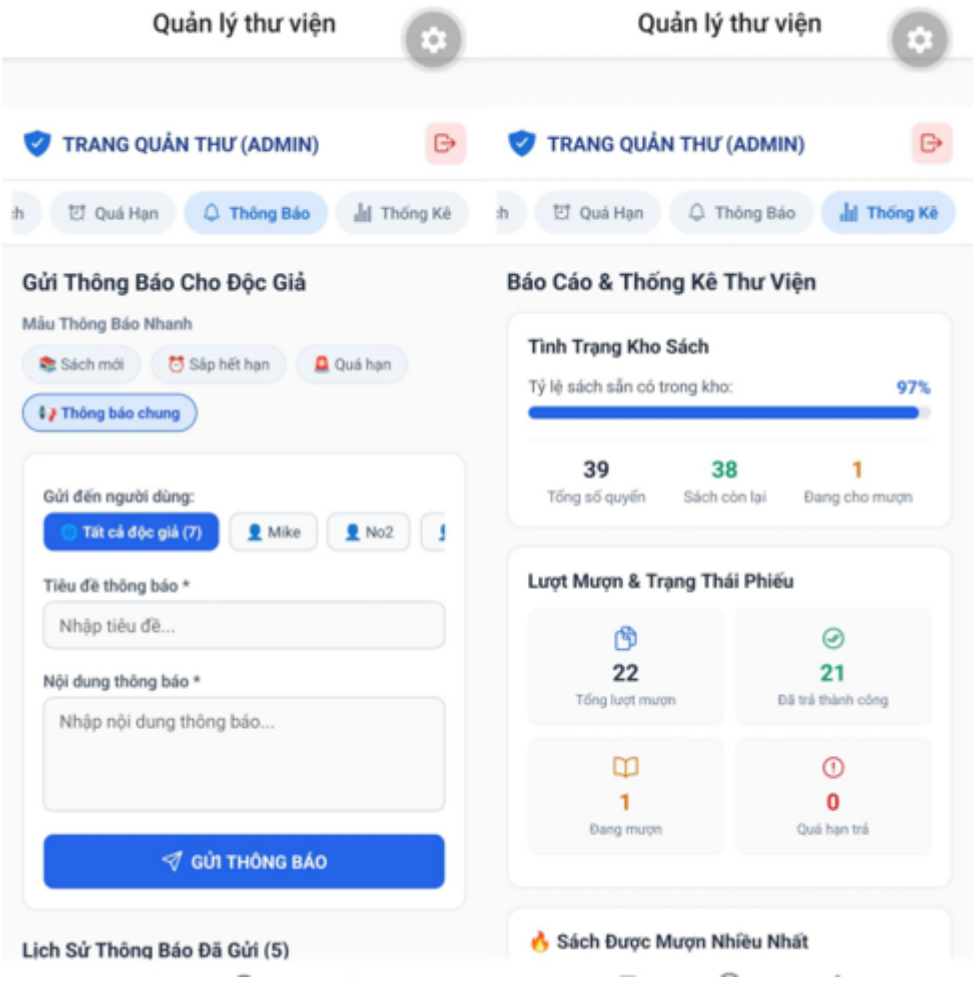
Đối với nghiệp vụ mượn sách, giao diện quản trị chia dữ liệu thành nhóm yêu cầu mượn và nhóm giao dịch đang mượn. Trong hình minh họa, màn hình ghi nhận 0 yêu cầu đang chờ và 1 giao dịch đang mượn, kèm thông tin sách, người mượn, ngày mượn và hạn trả.

Ở chức năng trả sách, hệ thống hiển thị tài liệu chưa được hoàn trả cùng thao tác xác nhận đã trả. Cách tổ chức này hỗ trợ quản thư thực hiện các bước xử lý theo trạng thái giao dịch, đồng thời tạo sự liên kết giữa hoạt động mượn và trả tài liệu.

Màn hình quản lý quá hạn cũng được xây dựng nhằm hỗ trợ xác định các trường hợp chưa hoàn trả sách đúng thời hạn. Trong ảnh chụp thực tế, danh sách quá hạn đang ở trạng thái trống và ứng dụng hiển thị thông báo không có trường hợp nào bị quá hạn. Đây là trạng

thái dữ liệu hiện tại, không đồng nghĩa với việc chức năng phát hiện quá hạn đã được kiểm thử đầy đủ trong mọi trường hợp.

Việc phân chia các nghiệp vụ theo trạng thái giúp quản thư dễ dàng theo dõi giao dịch và xác định những trường hợp cần xử lý, góp phần nâng cao tính rõ ràng trong hoạt động quản lý mượn/trả sách.



Hình 4.12. Giao diện gửi thông báo và thống kê tình trạng thư viện

e. Gửi thông báo cho độc giả

Chức năng thông báo được xây dựng nhằm cung cấp một kênh truyền đạt thông tin từ quản thư đến người sử dụng thư viện. Giao diện cho phép quản thư lựa chọn nhóm người nhận, nhập tiêu đề và nội dung thông báo trước khi thực hiện thao tác gửi.

Trong hình ảnh thực tế, ứng dụng cung cấp các mẫu thông báo như sách mới, sắp hết hạn, quá hạn và thông báo chung. Người quản lý có thể lựa chọn gửi đến tất cả độc giả hoặc các tài khoản cụ thể. Bên dưới biểu mẫu còn có khu vực lịch sử thông báo đã gửi, giúp theo dõi những nội dung đã được tạo trước đó.

Cách tổ chức này hỗ trợ quản thư truyền tải thông tin liên quan đến tài liệu và hoạt động thư viện mà không cần thông báo riêng cho từng người dùng. Đồng thời, việc lưu lại lịch sử thông báo tạo điều kiện cho quá trình tra cứu và quản lý nội dung đã phát hành.

Tuy nhiên, hình ảnh giao diện chưa đủ để xác nhận thông báo đã được phân phối thành công đến thiết bị nhận trong mọi trạng thái ứng dụng. Nội dung này cần được kiểm thử riêng nếu hệ thống sử dụng thông báo đẩy.

f. Báo cáo và thống kê thư viện

Chức năng thống kê được triển khai nhằm hỗ trợ quản thư theo dõi tình trạng sử dụng tài nguyên và tổng hợp hoạt động mượn/trả sách. Đây là thành phần quan trọng giúp chuyển dữ liệu giao dịch thành những thông tin có ý nghĩa đối với công tác quản lý.

Qua ảnh chụp màn hình thống kê, hệ thống hiển thị các chỉ số gồm tổng số 39 quyển sách, 38 quyển còn trong kho và 1 quyển đang được mượn, tương ứng tỷ lệ sách sẵn có khoảng 97%. Ngoài ra, giao diện ghi nhận 22 lượt mượn, 21 lượt đã trả thành công, 1 giao dịch đang mượn và 0 giao dịch quá hạn.

Các giá trị này có sự tương ứng về mặt số học tại thời điểm ghi nhận: 38 quyển còn trong kho và 1 quyển đang được mượn tạo thành tổng số 39 quyển; số lượt đã trả và đang mượn cũng cộng lại bằng tổng lượt mượn đang hiển thị. Đây là dấu hiệu cho thấy giao diện thống kê có khả năng tổng hợp các chỉ số liên quan để người quản lý theo dõi tình trạng thư viện.

Bên cạnh những chỉ số tổng quan, màn hình còn có khu vực thể hiện các đầu sách được mượn nhiều nhất, hỗ trợ nhận diện xu hướng sử dụng tài liệu nếu được cung cấp dữ liệu thống kê đầy đủ.

Nhìn chung, kết quả triển khai cho thấy ứng dụng đã xây dựng được giao diện thống kê tương đối trực quan, giúp quản thư theo dõi tình trạng sách và hoạt động mượn/trả thông qua các chỉ số tổng hợp. Tuy nhiên, những số liệu này chỉ phản ánh trạng thái dữ liệu trên giao diện tại thời điểm chụp, chưa phải bằng chứng về độ chính xác thống kê trong toàn bộ quá trình vận hành.

g. Đánh giá chung khu vực quản trị

Thông qua các ảnh chụp thực tế, có thể nhận thấy khu vực quản trị Smart Library đã xây dựng được hệ thống giao diện bao quát các nghiệp vụ chính của thư viện. Việc tổ chức chức năng theo từng nhóm như quản lý sách, người dùng, thẻ RFID, mượn/trả, quá hạn, thông báo và thống kê giúp công tác quản lý trở nên tập trung, có cấu trúc và thuận tiện hơn so với phương thức theo dõi thủ công.

Mỗi nhóm chức năng được triển khai thông qua các component tương ứng, trong khi màn hình librarian.tsx đảm nhiệm việc tập hợp và điều hướng giữa những khu vực quản trị. Cách tổ chức này phù hợp với quy mô của ứng dụng đồ án, đồng thời tạo điều kiện cho việc phát triển và bảo trì từng chức năng độc lập ở mức giao diện.

Tổng thể, các kết quả thể hiện trong mục 4.2 cho thấy Smart Library đã hình thành được một ứng dụng quản lý thư viện với hai nhóm người dùng, các màn hình nghiệp vụ và cơ chế kết nối dữ liệu Firebase. Đặc biệt, ứng dụng không chỉ cung cấp khả năng tra cứu, theo dõi mượn/trả dành cho độc giả mà còn hỗ trợ quản thư giám sát danh mục tài liệu, tài khoản, thẻ RFID và các số liệu hoạt động của thư viện.

Mặc dù vậy, những hình ảnh hiện có chủ yếu minh họa kết quả triển khai giao diện và một số trạng thái nghiệp vụ thực tế. Các nội dung như hiệu năng khi nhiều người truy cập đồng thời, độ tin cậy của giao dịch, khả năng xử lý khi mất mạng và độ ổn định của thiết bị RFID vẫn cần được đánh giá bằng những thử nghiệm riêng. Riêng các chức năng đặt chỗ ngồi và AI được đề cập trong mục tiêu ban đầu chưa có đủ minh chứng thực nghiệm trong nhóm hình hiện tại để đưa ra kết luận về mức độ hoàn thiện.

Những kết quả đã trình bày là cơ sở để tiếp tục đánh giá cơ chế đồng bộ dữ liệu, tính nhất quán nghiệp vụ, hiệu năng và các giới hạn kỹ thuật của hệ thống trong những mục tiếp theo của chương.

4.3 Đánh giá dữ liệu và cơ chế đồng bộ

4.3.1 Vai trò của Firebase Authentication và Realtime Database

Kiến trúc hiện tại dùng Firebase Authentication cho xác thực và Firebase Realtime Database cho dữ liệu nghiệp vụ như users, books và borrowRecords. AsyncStorage được dùng để duy trì persistence của phiên Firebase trên thiết bị. Sự phân chia này giúp tận dụng dịch vụ xác thực có sẵn, giảm nhu cầu tự xây dựng hệ thống quản lý mật khẩu và rút ngắn phần hạ tầng backend phải triển khai.

Realtime Database cung cấp listener onValue() để ứng dụng nhận thay đổi và cập nhật state. Cơ chế này phù hợp với dữ liệu có nhu cầu phản ánh nhanh, chẳng hạn danh sách sách hoặc lịch sử mượn. Tuy nhiên, việc dùng listener rộng trên một node lớn có thể khiến ứng dụng tải lại nhiều dữ liệu hơn cần thiết khi hệ thống phát triển. Cần xem xét phạm vi listener, cách tổ chức node và việc hủy listener khi component không còn sử dụng để hạn chế tiêu thụ băng thông và tài nguyên.

4.3.2 Tính nhất quán và an toàn dữ liệu

Một hệ thống thư viện phải duy trì sự nhất quán giữa số lượng sách, giao dịch mượn và giao dịch trả. Ví dụ, khi một giao dịch mượn thành công, số lượng còn lại và lịch sử mượn cần phản ánh cùng một kết quả nghiệp vụ. Nếu chỉ cập nhật một trong các phần dữ liệu, hệ thống có thể hiển thị tồn kho không chính xác hoặc mất dấu giao dịch. Vì vậy, cần thiết kế quy trình ghi dữ liệu có khả năng xử lý lỗi giữa chừng, dùng transaction hoặc cấu trúc dữ liệu phù hợp và có cơ chế đối soát khi cần.

Bảo mật cần được đánh giá độc lập với giao diện. Firebase Security Rules phải giới hạn quyền truy cập theo uid và role; người dùng thông thường chỉ được đọc/ghi dữ liệu phù hợp với quyền của họ, còn thao tác quản trị cần được bảo vệ chặt hơn. Các trường như role, số lượng tồn kho và trạng thái giao dịch không nên được tin cậy chỉ vì client gửi lên giá trị hợp lệ. Nên xác minh quy tắc ở lớp dữ liệu hoặc dịch vụ đáng tin cậy, đồng thời tránh lưu thông tin nhạy cảm không cần thiết trong hồ sơ người dùng.

4.4 Đánh giá hiệu năng và trải nghiệm người dùng

Do tài liệu đầu vào không có kết quả đo thời gian phản hồi hoặc tải thực nghiệm, phần này đánh giá tiềm năng và điểm cần đo thay vì đưa ra kết luận định lượng. Hiệu năng thực tế phụ thuộc vào số lượng sách, số người dùng đồng thời, chất lượng kết nối mạng, cấu hình thiết bị, số listener đang hoạt động và kích thước dữ liệu trả về từ Firebase.

4.4.1 Hiệu năng giao diện và tìm kiếm

React Native và Expo cho phép dùng chung phần lớn mã nguồn cho nhiều nền tảng, trong khi các màn hình được tổ chức thành component. Việc sử dụng FlatList ở màn hình danh sách sách là lựa chọn phù hợp với danh sách có thể dài vì danh sách có khả năng tối ưu việc hiển thị các phần tử nhìn thấy trên màn hình. Tìm kiếm cục bộ giảm số lần gọi mạng và có thể tạo cảm giác phản hồi nhanh khi tập dữ liệu ở quy mô vừa.

Dù vậy, không thể kết luận ứng dụng đạt một mức FPS hoặc thời gian phản hồi cụ thể khi chưa đo trên thiết bị. Khi số lượng sách lớn, cần theo dõi thời gian mở màn hình, độ trễ từ lúc nhập từ khóa đến khi kết quả cập nhật, mức sử dụng bộ nhớ và hiện tượng giật khi danh sách thay đổi. Có thể tối ưu bằng cách giới hạn số trường dữ liệu, memo hóa các component phù hợp, debounce tìm kiếm và tránh render lại không cần thiết.

4.4.2 Độ trễ đồng bộ dữ liệu

Cơ chế listener realtime giúp giao diện phản ánh dữ liệu từ Firebase mà không cần người dùng làm mới thủ công. Đây là lợi thế quan trọng đối với danh sách sách và lịch sử giao dịch. Tuy nhiên, realtime không đồng nghĩa với độ trễ bằng không: dữ liệu vẫn phụ thuộc vào kết nối mạng, trạng thái dịch vụ và cách ứng dụng đăng ký listener. Cần đo khoảng thời gian từ lúc ghi dữ liệu đến lúc màn hình khác nhận được thay đổi, đồng thời kiểm tra hành vi khi mạng yếu, mất mạng tạm thời hoặc ứng dụng chuyển nền rồi quay lại.

4.4.3 Độ ổn định và khả năng phục hồi

Một ứng dụng thư viện cần xử lý được lỗi xác thực, lỗi đọc/ghi dữ liệu, quyền camera bị từ chối và dữ liệu thiếu. Với mỗi yêu cầu bất đồng bộ, giao diện nên có trạng thái đang tải, thành công và thất bại; không nên để người dùng phải suy đoán liệu thao tác đã được ghi nhận hay chưa. Khi khôi phục kết nối, cần kiểm tra state trên thiết bị có được đồng bộ lại và các thao tác chưa hoàn thành có bị gửi lặp hay không.

Ngoài ra, persistence của phiên đăng nhập giúp trải nghiệm thuận tiện hơn khi mở lại ứng dụng, nhưng cũng đòi hỏi kiểm thử vòng đời phiên: khởi động nguội, đăng xuất, token hết hạn, tài khoản bị vô hiệu hóa và đổi tài khoản trên cùng thiết bị. Các trường hợp này có ý nghĩa trực tiếp đối với cả trải nghiệm và bảo mật.

4.4.4 Bộ chỉ số cần đo trong thực nghiệm

Chỉ số	Cách đo đề xuất	Ý nghĩa
Thời gian đăng nhập	Đo từ lúc nhấn đăng nhập đến khi vào màn hình đích; chạy nhiều lần trên cùng thiết bị và mạng	Phản ánh thời gian xác thực và tải hồ sơ
Thời gian mở danh sách sách	Đo từ lúc mở /books đến khi danh sách sẵn sàng thao tác	Phản ánh tải dữ liệu và render
Độ trễ tìm kiếm	Đo từ lúc nhập ký tự cuối đến khi kết quả ổn định	Đánh giá phản hồi tìm kiếm cục bộ

Độ trễ đồng bộ	Ghi thời điểm cập nhật ở một thiết bị và thời điểm thiết bị khác hiển thị	Đánh giá realtime trong điều kiện mạng cụ thể
Tỷ lệ ca kiểm thử đạt	Số ca đạt / tổng số ca đã thực hiện × 100%	Tóm tắt độ bao phủ và tính đúng đắn theo bộ ca
Tỷ lệ lỗi giao dịch	Số giao dịch lỗi hoặc trùng / tổng giao dịch thử	Đánh giá độ tin cậy của mượn/trả
Mức sử dụng tài nguyên	Theo dõi CPU/bộ nhớ bằng công cụ phù hợp trên thiết bị	Phát hiện render nặng hoặc rò rỉ listener

4.5 Kế hoạch kiểm thử chức năng chi tiết

Bảng dưới đây là bộ ca kiểm thử đề xuất dựa trên các chức năng và luồng được mô tả trong tài liệu. Cột "Kết quả thực tế" cần được nhóm cập nhật sau khi chạy trên bản ứng dụng. Không nên đánh dấu tất cả ca là đạt khi chưa có bằng chứng thực nghiệm.

Mã	Chức năng	Dữ liệu/kịch bản	Kết quả mong đợi
TC01	Đăng nhập với tài khoản hợp lệ	Email/mật khẩu hợp lệ; hồ sơ có role=user	Đăng nhập thành công và điều hướng /home
TC02	Đăng nhập sai mật khẩu	Email tồn tại, mật khẩu sai	Không đăng nhập; thông báo lỗi rõ ràng
TC03	Hồ sơ thiếu role	Auth hợp lệ nhưng users/{uid} thiếu role	Không cấp quyền mặc định; báo lỗi hoặc yêu cầu hoàn thiện hồ sơ
TC04	Phân quyền thủ thư	Tài khoản role=librarian	Điều hướng /librarian; dữ liệu quản trị được bảo vệ
TC05	Truy cập quản trị trái phép	Tài khoản user thử mở route hoặc đọc node quản trị	Bị chặn ở cả giao diện và Security Rules
TC06	Đăng ký hợp lệ	Email chưa đăng ký và mật khẩu đáp ứng quy tắc	Tạo Auth user và hồ sơ ứng dụng
TC07	Đăng ký email trùng	Email đã tồn tại	Không tạo tài khoản trùng; thông báo phù hợp

TC08	Tìm theo tiêu đề	Từ khóa nằm trong tiêu đề sách	Hiển thị các sách khớp từ khóa
TC09	Tìm theo tác giả/ISBN/vị trí	Từ khóa khớp một trường không phải tiêu đề	Kết quả đúng theo trường được hỗ trợ
TC10	Tìm kiếm rỗng/không khớp	Chuỗi rỗng hoặc từ khóa không tồn tại	Hiển thị danh sách mặc định hoặc trạng thái không có kết quả
TC11	Mở chi tiết sách	Chọn sách có id hợp lệ	Hiển thị đúng thông tin của sách
TC12	Id sách không tồn tại	Mở route với id không hợp lệ	Hiển thị trạng thái không tìm thấy, không crash
TC13	Mượn số lượng hợp lệ	Tồn kho đủ; quantity là số nguyên dương	Tạo bản ghi mượn và giảm tồn kho đúng
TC14	Mượn số lượng bằng 0/âm	quantity=0 hoặc quantity<0	Từ chối thao tác
TC15	Mượn vượt tồn kho	quantity lớn hơn số lượng còn lại	Từ chối; tồn kho không âm
TC16	Hai người mượn đồng thời	Hai yêu cầu cùng mượn số lượng cuối cùng	Không vượt tồn kho; giao dịch nhất quán
TC17	Trả sách hợp lệ	Giao dịch đang được mượn	Cập nhật trạng thái trả và tồn kho đúng một lần
TC18	Trả sách lặp lại	Gửi lại yêu cầu trả cùng giao dịch	Không cộng tồn kho hoặc tạo bản ghi trả trùng
TC19	Quét mã hợp lệ	Barcode/QR tồn tại trong danh mục	Hiển thị đúng sách và cho phép tiếp tục
TC20	Quét mã không tồn tại	Barcode/QR không có trong danh mục	Thông báo không tìm thấy sách
TC21	Từ chối quyền camera	Người dùng không cấp quyền camera	Giải thích quyền cần thiết và hướng dẫn xử lý
TC22	Mất mạng khi ghi giao dịch	Ngắt mạng ngay lúc mượn/trả	Thông báo thất bại/trạng thái rõ; không ghi dữ liệu lệch
TC23	Đồng bộ trên hai thiết bị	Cùng mở danh sách/lịch sử; một thiết bị cập nhật	Thiết bị còn lại nhận dữ liệu theo cơ chế listener
TC24	Thêm/sửa/xóa sách	Tài khoản thủ thư nhập dữ liệu hợp lệ	Dữ liệu được lưu và hiển thị nhất quán

TC25	Dữ liệu quản trị không hợp lệ	Thiếu tên sách, số lượng âm hoặc mã trùng	Validation ngăn lưu dữ liệu không hợp lệ

- Chuẩn bị môi trường: ghi rõ phiên bản ứng dụng, thiết bị, hệ điều hành, phiên bản Expo/React Native và cấu hình Firebase được dùng.
- Tạo bộ dữ liệu kiểm thử có kiểm soát: sách còn nhiều bản, sách chỉ còn một bản, sách hết hàng, tài khoản user, tài khoản librarian và hồ sơ thiếu trường.
- Thực hiện từng ca theo đúng tiền điều kiện; ghi lại đầu vào, thời điểm chạy, kết quả quan sát và ảnh chụp màn hình nếu cần.
- Với ca mượn/trả, kiểm tra đồng thời cả giao diện, node books và borrowRecords trong Firebase để xác nhận dữ liệu không chỉ hiển thị đúng mà còn được ghi đúng.
- Lặp lại các ca quan trọng nhiều lần, đặc biệt là đăng nhập, mượn/trả, đồng bộ realtime và xử lý mất mạng.
- Phân loại lỗi theo mức độ: nghiêm trọng (mất/sai dữ liệu, vượt tồn kho, vượt quyền), cao (chức năng chính không sử dụng được), trung bình (sai trạng thái hoặc thông báo), thấp (lỗi trình bày).
- Sau khi sửa lỗi, chạy lại ca liên quan và kiểm thử hồi quy để bảo đảm thay đổi không làm hỏng chức năng đã hoạt động.

4.6 Đối chiếu với mục tiêu ban đầu

Mục tiêu ban đầu của dự án bao gồm xây dựng ứng dụng di động quản lý thư viện, hỗ trợ tìm kiếm sách, mượn/trả, lịch sử và chức năng quản trị; đồng thời định hướng tích hợp IoT, đặt chỗ ngồi theo thời gian thực và module AI tùy chọn. Tuy nhiên, Chương 3 mô tả source code hiện tại chủ yếu xoay quanh ứng dụng mobile, Firebase Authentication, Realtime Database, dữ liệu sách, lịch sử mượn/trả và quét mã. Vì vậy, mức độ đáp ứng cần được phân loại theo bằng chứng hiện có thay vì gộp mọi ý tưởng trong phạm vi tổng thể thành chức năng đã hoàn thành.

Yêu cầu/mục tiêu	Bằng chứng trong tài liệu	Đánh giá thận trọng
Ứng dụng đa nền tảng	React Native, Expo, TypeScript	Có nền tảng kỹ thuật phù hợp; chưa có kết quả build/kiểm thử trên từng nền tảng
Đăng nhập/đăng ký	Firebase Authentication; index.tsx, register.tsx	Luồng xử lý được mô tả rõ ở mức kiến trúc
Phân quyền user/librarian	Đọc role từ users/{uid}, điều hướng /home hoặc /librarian	Có cơ chế điều hướng theo role; mức bảo mật phụ thuộc Security Rules
Tìm kiếm và chi tiết sách	books.tsx, book-detail.tsx, LibraryContext	Được mô tả là đã có luồng tìm kiếm cục bộ và mở chi tiết
Mượn/trả và lịch sử	borrowBook(), returnBook(), borrowRecords	Có các thành phần nghiệp vụ chính trong kiến trúc
Quét barcode/QR	scan-book.tsx, expo-camera	Có luồng đọc mã và đối chiếu với danh sách sách
Quản trị thư viện	Các component AdminBookManager, AdminUserManager, AdminBorrowManager...	Có cấu trúc component theo nhóm nghiệp vụ
Đặt chỗ ngồi qua ESP32	Được nêu trong mục tiêu tổng thể, nhưng không nằm trong luồng source mobile được phân tích ở Chương 3	Chưa đủ bằng chứng để xác nhận đã tích hợp trong phiên bản source được mô tả
AI gợi ý sách/dự đoán chỗ ngồi	Được giới thiệu là module tùy chọn ở Chương 1–2	Chưa có bằng chứng thực nghiệm về mô hình đã huấn luyện hoặc API AI hoạt động
Thông báo đẩy	Được đề cập trong phạm vi thiết kế tổng thể	Chưa đủ bằng chứng từ phần source được mô tả để kết luận đã triển khai

Từ bảng đối chiếu có thể thấy phần có bằng chứng mô tả rõ nhất là ứng dụng di động phục vụ tài khoản, danh mục sách, chi tiết sách, mượn/trả, lịch sử, quét mã và khu vực quản trị. Các thành phần IoT, đặt chỗ ngồi, AI và thông báo cần được xác minh thêm bằng source code tương ứng, bản demo hoặc kết quả thực nghiệm.

4.7 Hạn chế phát hiện và hướng cải thiện

4.7.1 Hạn chế về kiến trúc phần mềm

Kiến trúc hiện tại là component-based của React Native, kết hợp Expo Router, React Context và Firebase-backed data access. Chưa có các lớp Repository, Service hoặc UseCase tách biệt. Một số component vừa hiển thị giao diện vừa xử lý validation và gọi Firebase trực tiếp. Cách tổ chức này phù hợp với quy mô đồ án ban đầu nhưng có thể khiến logic khó tái sử dụng, khó viết kiểm thử đơn vị và khó thay đổi nhà cung cấp dữ liệu.

Hướng cải thiện là đưa các thao tác đọc/ghi Firebase vào các service hoặc repository thống nhất; chuyển các quy tắc nghiệp vụ mượn/trả sang hàm dùng chung có thể kiểm thử độc lập; và giữ component chủ yếu chịu trách nhiệm trình bày, tiếp nhận tương tác và hiển thị trạng thái. Việc phân lớp nên được thực hiện từng bước để tránh tái cấu trúc quá lớn trong khi chưa có bộ kiểm thử hồi quy.

4.7.2 Hạn chế về độ tin cậy của giao dịch

Mượn/trả là nghiệp vụ ảnh hưởng trực tiếp đến tồn kho và lịch sử. Nếu chỉ kiểm tra số lượng ở client, hệ thống có thể gặp lỗi khi nhiều yêu cầu cùng lúc hoặc khi kết nối bị ngắt giữa các lần ghi. Cần dùng thao tác nguyên tử phù hợp, kiểm tra quyền ghi, chống gửi lặp và thiết kế trạng thái giao dịch rõ ràng. Với các giao dịch quan trọng, nên có cách đối soát giữa số lượng sách và các bản ghi đang mượn.

4.7.3 Hạn chế về bảo mật

Việc chuyển người dùng đến màn hình theo trường role là một phần của trải nghiệm điều hướng, không thay thế cơ chế kiểm soát quyền truy cập dữ liệu. Cần rà soát Firebase Security Rules cho từng node; giới hạn người dùng chỉ sửa hồ sơ của chính mình; giới hạn quyền quản trị; xác thực tính hợp lệ của các trường nhạy cảm như role, tồn kho và trạng

thái giao dịch. Cần kiểm tra cả tình huống người dùng gọi trực tiếp SDK hoặc cố truy cập dữ liệu ngoài giao diện thông thường.

4.7.4 Hạn chế về bằng chứng kiểm thử và hiệu năng

Tài liệu hiện có mô tả kiến trúc và luồng xử lý nhưng chưa kèm bảng kết quả kiểm thử, log lỗi, số liệu độ trễ hoặc kết quả tải đồng thời. Vì vậy, chưa thể khẳng định hệ thống đạt một tỷ lệ thành công hay mức hiệu năng cụ thể. Để tăng tính thuyết phục của báo cáo, nhóm cần bổ sung kết quả kiểm thử thực tế, ghi rõ môi trường chạy và đính kèm bằng chứng cho các chức năng cốt lõi.

4.7.5 Hạn chế về phạm vi IoT và AI

Smart Library được định hướng là hệ thống kết hợp ứng dụng mobile, ESP32, cảm biến và AI tùy chọn. Tuy nhiên, phần kiến trúc source mobile hiện được mô tả tập trung vào quản lý sách và Firebase Realtime Database; chưa có kết quả đo từ cảm biến ghế, nhiệt độ/độ ẩm, ánh sáng hoặc chỉ số đánh giá mô hình AI. Nếu nhóm có triển khai các thành phần này, nên bổ sung sơ đồ đấu nối, luồng dữ liệu thiết bị, ảnh demo, log gửi nhận, bộ dữ liệu dùng cho AI và chỉ số đánh giá thích hợp. Nếu chưa triển khai, nên trình bày rõ đây là phạm vi định hướng hoặc công việc tương lai.

4.7.6 Lộ trình cải thiện theo mức ưu tiên

Mức ưu tiên	Việc cải thiện	Lý do
P0 – Bắt buộc	Kiểm thử và củng cố luồng mượn/trả; bảo vệ Security Rules	Tránh sai tồn kho, giao dịch trùng và truy cập trái phép
P1 – Cao	Bổ sung trạng thái loading/error/empty nhất quán	Giúp người dùng hiểu trạng thái thao tác và phục hồi lỗi

P1 – Cao	Tách Firebase access khỏi component quản trị	Giảm phụ thuộc và tăng khả năng kiểm thử
P2 – Trung bình	Đo hiệu năng trên thiết bị và danh mục dữ liệu lớn	Xác định giới hạn thực tế thay vì suy đoán
P2 – Trung bình	Tối ưu listener và truy vấn dữ liệu	Giảm tải mạng và render không cần thiết
P3 – Theo phạm vi	Xác minh hoặc hoàn thiện IoT, đặt chỗ và AI	Đồng bộ sản phẩm với mục tiêu hệ thống thông minh

4.8 Kết luận chương

Chương 4 đã phân tích kết quả triển khai Smart Library dựa trên cấu trúc source code và các luồng nghiệp vụ được trình bày ở Chương 3. Ứng dụng đã được mô tả với những thành phần chính gồm React Native/Expo/TypeScript, Expo Router, Firebase Authentication, Firebase Realtime Database, AsyncStorage và LibraryContext. Các luồng đăng nhập, đăng ký, tìm kiếm sách, xem chi tiết, mượn/trả, quét mã và các component quản trị tạo thành nền tảng chức năng cho ứng dụng quản lý thư viện.

Về ưu điểm, việc sử dụng Firebase giúp giảm khối lượng hạ tầng backend phải tự xây dựng; LibraryContext hỗ trợ chia sẻ dữ liệu và logic; Expo Router làm rõ cấu trúc điều hướng; còn việc chia component quản trị theo nghiệp vụ giúp tổ chức giao diện tương đối rõ ràng. Cơ chế listener realtime cũng tạo điều kiện cập nhật giao diện khi dữ liệu thay đổi mà không yêu cầu người dùng làm mới thủ công.

CHƯƠNG 5: KẾT LUẬN

5.1 Tổng kết những kết quả đạt được

Trải qua quá trình nghiên cứu, phân tích và phát triển, đồ án "Smart Library - Ứng dụng quản lý thư viện và đặt chỗ ngồi thông minh" đã hoàn thành tốt các mục tiêu đề ra ban đầu. Nhóm thực hiện đã xây dựng thành công một hệ thống quản lý thư viện toàn diện với những kết quả cụ thể như sau:

- Về mặt ứng dụng: Đã phát triển thành công ứng dụng di động đa nền tảng (Android/iOS) bằng React Native và TypeScript. Ứng dụng cung cấp giao diện trực quan, mượt mà và đầy đủ các tính năng thiết yếu cho sinh viên (tìm kiếm sách, đặt chỗ ngồi, mượn/trả, xem lịch sử) cũng như công cụ quản trị mạnh mẽ cho thủ thư (quản lý sách, người dùng, theo dõi trạng thái mượn/trả).
- Về mặt phần cứng IoT: Triển khai thành công vi điều khiển ESP32 cùng cảm biến RFID để thu thập dữ liệu môi trường và trạng thái chỗ ngồi thực tế tại thư viện.
- Về mặt hệ thống Backend: Ứng dụng xuất sắc các dịch vụ của nền tảng đám mây Google Firebase (Authentication, Realtime Database/Firestore). Hệ thống đảm bảo được tính đồng bộ dữ liệu thời gian thực giữa ứng dụng di động và phần cứng IoT, mang lại trải nghiệm liền mạch cho người dùng.

5.2 Những hạn chế hiện tại

Bên cạnh những kết quả tích cực, do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, hệ thống Smart Library vẫn còn tồn tại một số hạn chế nhất định:

- Về kiến trúc phần mềm: Như đã phân tích, ứng dụng hiện đang áp dụng kiến trúc component-based kết hợp Context-based state management. Ranh giới giữa lớp giao diện (Presentation), logic nghiệp vụ (Business Logic) và truy cập dữ liệu (Data

Access) chưa hoàn toàn được tách biệt độc lập, gây khó khăn cho việc bảo trì và mở rộng nếu quy mô dự án lớn hơn.

- Về giới hạn phần cứng: Việc triển khai thực tế trên diện rộng sẽ đòi hỏi giải pháp tối ưu hóa mạng lưới kết nối giữa hàng trăm/hàng nghìn thiết bị ESP32, cũng như xử lý nhiễu tín hiệu cảm biến trong môi trường thực.

5.3 Hướng phát triển tương lai

Nhằm hoàn thiện và nâng cao giá trị thực tiễn của sản phẩm, nhóm đề xuất các hướng phát triển tương lai cho hệ thống Smart Library như sau:

- **Tái cấu trúc mã nguồn (Refactoring):** Chuyển đổi dần kiến trúc hiện tại sang các mô hình chuẩn hóa và rõ ràng hơn như Clean Architecture hoặc MVVM. Tách biệt các thao tác gọi Firebase vào các lớp Repository/Service riêng lẻ để dễ dàng viết test và quản lý.
- **Phát triển Module AI:** Thu thập thêm dữ liệu thực tế để huấn luyện các mô hình Machine Learning, từ đó chính thức đưa hệ thống gợi ý sách (Recommendation System) và dự đoán chỗ ngồi vào hoạt động thực tiễn.
- **Mở rộng tính năng thanh toán:** Tích hợp các cổng thanh toán điện tử (như MoMo, VNPay) để hỗ trợ sinh viên thanh toán các khoản phí phạt trả sách quá hạn một cách tiện lợi.
- **Nâng cấp thẻ thư viện số:** Tích hợp công nghệ NFC trên điện thoại thông minh thay thế cho đầu đọc thẻ RFID truyền thống, giúp người dùng chạm điện thoại để mượn sách hoặc check-in chỗ ngồi trực tiếp.

Với những kết quả đã đạt được, dự án Smart Library không chỉ là một bài tập thực hành môn học mang tính học thuật mà còn mở ra tiềm năng ứng dụng rất lớn, góp phần thúc đẩy công cuộc chuyển đổi số và xây dựng môi trường học tập hiện đại tại các trường Đại học.

LINK GITHUB VÀ VIDEO DEMO

- Link github project: https://github.com/nt131513-tech/SmartLibrary/tree/d0edd350c1ea90e0f5a16b78de675e2b9dfa3ab0/NHOM8_TONG_H%E1%BB%A2P_CK

- Link video demo: https://youtu.be/-57BBjQVFCY?si=SSpwRWx22WjrUXHm

- link full dự án: https://github.com/nt131513-tech/SmartLibrary.git